```
cloudra/__init__.py:1   """Cloudra - AI Agent collaboration framework for chemical research."""
cloudra/__init__.py:2
cloudra/__init__.py:3   from importlib import import_module
cloudra/__init__.py:4
cloudra/__init__.py:5   __version__ = "2.0.0-beta"
cloudra/__init__.py:6
cloudra/__init__.py:7   _LAZY_EXPORTS = {
cloudra/__init__.py:8       "Agent": ("cloudra.core.agent", "Agent"),
cloudra/__init__.py:9       "Coordinator": ("cloudra.core.agent", "Coordinator"),
cloudra/__init__.py:10  }
cloudra/__init__.py:11  # 兼容垫片：包结构重排（core/services/persistence/base/infra）前的
cloudra/__init__.py:12  # `from cloudra import X` 老路径全部继续可用（惰性加载，不提前拉起重依赖）。
cloudra/__init__.py:13  _LAZY_MODULES = {
cloudra/__init__.py:14      # core
cloudra/__init__.py:15      "agent": "cloudra.core.agent",
cloudra/__init__.py:16      "supervisor": "cloudra.core.supervisor",
cloudra/__init__.py:17      # services
cloudra/__init__.py:18      "session_store": "cloudra.services.session_store",
cloudra/__init__.py:19      "nodes": "cloudra.services.nodes",
cloudra/__init__.py:20      # persistence
cloudra/__init__.py:21      "database": "cloudra.persistence.database",
cloudra/__init__.py:22      "artifacts": "cloudra.persistence.artifacts",
cloudra/__init__.py:23      "memory": "cloudra.persistence.memory",
cloudra/__init__.py:24      "context_manager": "cloudra.persistence.context_manager",
cloudra/__init__.py:25      "paths": "cloudra.persistence.paths",
cloudra/__init__.py:26      "fs_utils": "cloudra.persistence.fs_utils",
cloudra/__init__.py:27      "workspace": "cloudra.persistence.workspace",
cloudra/__init__.py:28      # base
cloudra/__init__.py:29      "session_class": "cloudra.base.session_class",
cloudra/__init__.py:30      "conversation_class": "cloudra.base.conversation_class",
cloudra/__init__.py:31      "log_class": "cloudra.base.log_class",
cloudra/__init__.py:32      "skill_class": "cloudra.base.skill_class",
cloudra/__init__.py:33      "tools_class": "cloudra.base.tools_class",
cloudra/__init__.py:34      # infra
cloudra/__init__.py:35      "config": "cloudra.infra.config",
cloudra/__init__.py:36      "llm_client": "cloudra.infra.llm_client",
cloudra/__init__.py:37      "events": "cloudra.infra.events",
cloudra/__init__.py:38      "ipc": "cloudra.infra.ipc",
cloudra/__init__.py:39      "streaming": "cloudra.infra.streaming",
cloudra/__init__.py:40      "token_estimator": "cloudra.infra.token_estimator",
cloudra/__init__.py:41      "web_search_native": "cloudra.infra.web_search_native",
cloudra/__init__.py:42      "slurm": "cloudra.infra.slurm",
cloudra/__init__.py:43      "compute_utils": "cloudra.infra.compute_utils",
cloudra/__init__.py:44      "operations": "cloudra.infra.operations",
cloudra/__init__.py:45      "yolo_guard": "cloudra.infra.yolo_guard",
cloudra/__init__.py:46      "platform_utils": "cloudra.infra.platform_utils",
cloudra/__init__.py:47      "runtime_profile": "cloudra.infra.runtime_profile",
cloudra/__init__.py:48  }
cloudra/__init__.py:49
cloudra/__init__.py:50  __all__ = [*_LAZY_EXPORTS, *_LAZY_MODULES, "__version__"]
cloudra/__init__.py:51
cloudra/__init__.py:52
```

```
cloudra/__init__.py:53  def __getattr__(name: str):
cloudra/__init__.py:54      """Keep legacy package exports without importing the Agent runtime eagerly."""
cloudra/__init__.py:55      target = _LAZY_EXPORTS.get(name)
cloudra/__init__.py:56      if target is not None:
cloudra/__init__.py:57          module_name, attribute = target
cloudra/__init__.py:58          value = getattr(import_module(module_name), attribute)
cloudra/__init__.py:59          globals()[name] = value
cloudra/__init__.py:60          return value
cloudra/__init__.py:61      module_path = _LAZY_MODULES.get(name)
cloudra/__init__.py:62      if module_path is not None:
cloudra/__init__.py:63          value = import_module(module_path)
cloudra/__init__.py:64          globals()[name] = value
cloudra/__init__.py:65          return value
cloudra/__init__.py:66      raise AttributeError(f"module 'cloudra' has no attribute {name!r}")
cloudra/__init__.py:67
cloudra/__init__.py:68
cloudra/__init__.py:69  def __dir__() -> list[str]:
cloudra/__init__.py:70      return sorted(set(globals()) | set(__all__))
cloudra/__main__.py:1   """Allow running as `python -m cloudra`."""
cloudra/__main__.py:2   from cloudra.cli import main
cloudra/__main__.py:3
cloudra/__main__.py:4   if __name__ == "__main__":
cloudra/__main__.py:5       main()
cloudra/base/__init__.py:1  """cloudra.base 子包。"""
cloudra/base/conversation_class.py:1    import json
cloudra/base/conversation_class.py:2    import os
cloudra/base/conversation_class.py:3    import re
cloudra/base/conversation_class.py:4    import uuid
cloudra/base/conversation_class.py:5    from datetime import datetime
cloudra/base/conversation_class.py:6
cloudra/base/conversation_class.py:7    from cloudra.persistence import paths
cloudra/base/conversation_class.py:8    from cloudra.persistence.fs_utils import atomic_write_json
cloudra/base/conversation_class.py:9
cloudra/base/conversation_class.py:10
cloudra/base/conversation_class.py:11   class Conversation:
cloudra/base/conversation_class.py:12       """
cloudra/base/conversation_class.py:13       用户级会话容器，隔离不同对话的上下文和历史记录。
cloudra/base/conversation_class.py:14       一个 Conversation 包含：
cloudra/base/conversation_class.py:15         - contexts/   : 该会话下各 agent 的上下文文件
cloudra/base/conversation_class.py:16         - sessions/   : 该会话下 reAct 执行记录
cloudra/base/conversation_class.py:17         - meta.json   : 会话元数据（创建时间、最后活跃时间、状态）
cloudra/base/conversation_class.py:18
cloudra/base/conversation_class.py:19       状态说明：
cloudra/base/conversation_class.py:20         - active  : 新建或刚恢复，尚未开始运行
cloudra/base/conversation_class.py:21         - running : Coordinator 正在运行中
cloudra/base/conversation_class.py:22         - sleeping: Coordinator 已完成任务，处于休眠等待唤醒状态
cloudra/base/conversation_class.py:23         - archived: 已归档，不再活跃
cloudra/base/conversation_class.py:24       """
cloudra/base/conversation_class.py:25
cloudra/base/conversation_class.py:26       id: str
cloudra/base/conversation_class.py:27       path: str
cloudra/base/conversation_class.py:28       create_time: datetime
```

```
cloudra/base/conversation_class.py:29       last_active: datetime
cloudra/base/conversation_class.py:30       state: str = 'active'
cloudra/base/conversation_class.py:31
cloudra/base/conversation_class.py:32       VALID_STATES = {'active', 'running', 'sleeping', 'archived'}
cloudra/base/conversation_class.py:33       ID_PATTERN = re.compile(r"^[A-Za-z0-9_-]{1,100}$")
cloudra/base/conversation_class.py:34
cloudra/base/conversation_class.py:35       def __init__(self, conv_id: str = None):
cloudra/base/conversation_class.py:36           if conv_id:
cloudra/base/conversation_class.py:37               if not self.ID_PATTERN.fullmatch(conv_id):
cloudra/base/conversation_class.py:38                   raise ValueError("会话 ID 只能包含字母、数字、下划线和连字符，长度不超过
100")
cloudra/base/conversation_class.py:39               # 恢复已有对话
cloudra/base/conversation_class.py:40               self.id = conv_id
cloudra/base/conversation_class.py:41               self.path = str(paths.conversation_dir(conv_id))
cloudra/base/conversation_class.py:42               self._load_meta()
cloudra/base/conversation_class.py:43           else:
cloudra/base/conversation_class.py:44               # 新建对话
cloudra/base/conversation_class.py:45               self.id = self._generate_id()
cloudra/base/conversation_class.py:46               self.path = str(paths.conversation_dir(self.id))
cloudra/base/conversation_class.py:47               self.create_time = datetime.now()
cloudra/base/conversation_class.py:48               self.last_active = self.create_time
cloudra/base/conversation_class.py:49               self.state = 'active'
cloudra/base/conversation_class.py:50               self._ensure_dirs()
cloudra/base/conversation_class.py:51               self._save_meta()
cloudra/base/conversation_class.py:52
cloudra/base/conversation_class.py:53       def _generate_id(self) -> str:
cloudra/base/conversation_class.py:54           return f"conv_{datetime.now().strftime('%Y%m%d_%H%M%S')}_{uuid.uuid4().hex
[:6]}"
cloudra/base/conversation_class.py:55
cloudra/base/conversation_class.py:56       def _ensure_dirs(self):
cloudra/base/conversation_class.py:57           os.makedirs(os.path.join(self.path, 'contexts'), exist_ok=True)
cloudra/base/conversation_class.py:58           os.makedirs(os.path.join(self.path, 'sessions'), exist_ok=True)
cloudra/base/conversation_class.py:59
cloudra/base/conversation_class.py:60       def _meta_path(self) -> str:
cloudra/base/conversation_class.py:61           return os.path.join(self.path, 'meta.json')
cloudra/base/conversation_class.py:62
cloudra/base/conversation_class.py:63       def _save_meta(self):
cloudra/base/conversation_class.py:64           atomic_write_json(
cloudra/base/conversation_class.py:65               self._meta_path(),
cloudra/base/conversation_class.py:66               {
cloudra/base/conversation_class.py:67                   'id': self.id,
cloudra/base/conversation_class.py:68                   'create_time': self.create_time.isoformat(),
cloudra/base/conversation_class.py:69                   'last_active': self.last_active.isoformat(),
cloudra/base/conversation_class.py:70                   'state': self.state,
cloudra/base/conversation_class.py:71               },
cloudra/base/conversation_class.py:72           )
cloudra/base/conversation_class.py:73
cloudra/base/conversation_class.py:74       def _load_meta(self):
cloudra/base/conversation_class.py:75           meta_path = self._meta_path()
cloudra/base/conversation_class.py:76           if os.path.exists(meta_path):
cloudra/base/conversation_class.py:77               with open(meta_path, 'r', encoding='utf-8') as f:
cloudra/base/conversation_class.py:78                   meta = json.load(f)
cloudra/base/conversation_class.py:79                   self.create_time = datetime.fromisoformat(meta['create_time'])
cloudra/base/conversation_class.py:80                   self.last_active = datetime.fromisoformat(meta['last_active'])
```

```
cloudra/base/conversation_class.py:81                    self.state = meta.get('state', 'active')
cloudra/base/conversation_class.py:82            else:
cloudra/base/conversation_class.py:83                self.create_time = datetime.now()
cloudra/base/conversation_class.py:84                self.last_active = self.create_time
cloudra/base/conversation_class.py:85                self.state = 'active'
cloudra/base/conversation_class.py:86
cloudra/base/conversation_class.py:87        def touch(self):
cloudra/base/conversation_class.py:88            """更新最后活跃时间并持久化。"""
cloudra/base/conversation_class.py:89            self.last_active = datetime.now()
cloudra/base/conversation_class.py:90            self._ensure_dirs()
cloudra/base/conversation_class.py:91            self._save_meta()
cloudra/base/conversation_class.py:92
cloudra/base/conversation_class.py:93        def set_state(self, state: str):
cloudra/base/conversation_class.py:94            """更新会话状态，仅接受预定义状态。"""
cloudra/base/conversation_class.py:95            if state not in self.VALID_STATES:
cloudra/base/conversation_class.py:96                raise ValueError(f"无效的会话状态: {state}，必须是 {self.VALID_STATES} 之一
")
cloudra/base/conversation_class.py:97            self.state = state
cloudra/base/conversation_class.py:98            self.touch()
cloudra/base/conversation_class.py:99
cloudra/base/conversation_class.py:100       def get_context_path(self, agent_id: str) -> str:
cloudra/base/conversation_class.py:101           return os.path.join(self.path, 'contexts', f'{agent_id}.json')
cloudra/base/conversation_class.py:102
cloudra/base/conversation_class.py:103       def get_session_path(self, session_id: str) -> str:
cloudra/base/conversation_class.py:104           return os.path.join(self.path, 'sessions', f'{session_id}.json')
cloudra/base/conversation_class.py:105
cloudra/base/conversation_class.py:106       def list_contexts(self) -> list[str]:
cloudra/base/conversation_class.py:107           """列出该会话下所有已有的 agent context 文件。"""
cloudra/base/conversation_class.py:108           ctx_dir = os.path.join(self.path, 'contexts')
cloudra/base/conversation_class.py:109           if not os.path.exists(ctx_dir):
cloudra/base/conversation_class.py:110               return []
cloudra/base/conversation_class.py:111           return [f for f in os.listdir(ctx_dir) if f.endswith('.json')]
cloudra/base/conversation_class.py:112
cloudra/base/conversation_class.py:113       def archive(self):
cloudra/base/conversation_class.py:114           """归档会话，不再活跃。"""
cloudra/base/conversation_class.py:115           self.set_state('archived')
cloudra/base/log_class.py:1 import os
cloudra/base/log_class.py:2
cloudra/base/log_class.py:3 from cloudra.base.session_class import Session
cloudra/base/log_class.py:4 from cloudra.infra.config import get_config
cloudra/base/log_class.py:5
cloudra/base/log_class.py:6
cloudra/base/log_class.py:7 class Log:
cloudra/base/log_class.py:8     '''
cloudra/base/log_class.py:9     LOG类用于记录和管理reAct的日志信息，包括推理过程、行动结果和观察结果等。
cloudra/base/log_class.py:10        '''
cloudra/base/log_class.py:11        name: str = 'default.log'
cloudra/base/log_class.py:12        session: Session
cloudra/base/log_class.py:13        level: int
cloudra/base/log_class.py:14        is_main: bool
cloudra/base/log_class.py:15        path: str
cloudra/base/log_class.py:16        # 类级开关：TUI 模式置 False，终端输出改由 TUI 事件渲染器负责；
cloudra/base/log_class.py:17        # 日志文件写入不受此开关影响。
```

```
cloudra/base/log_class.py:18        terminal_output: bool = True
cloudra/base/log_class.py:19
cloudra/base/log_class.py:20        def __init__(self, log_name: str = 'default.log', log_path: str = None):
cloudra/base/log_class.py:21            self.name = log_name
cloudra/base/log_class.py:22            if log_path:
cloudra/base/log_class.py:23                self.path = log_path
cloudra/base/log_class.py:24            else:
cloudra/base/log_class.py:25                self.path = os.path.join(get_config().data_dir, 'logs', log_name)
cloudra/base/log_class.py:26            self.session = None
cloudra/base/log_class.py:27            self.level = 0
cloudra/base/log_class.py:28            self.is_main = False
cloudra/base/log_class.py:29            # 流式输出期间持有的文件句柄（见 begin_stream/end_stream）
cloudra/base/log_class.py:30            self._stream_file = None
cloudra/base/log_class.py:31
cloudra/base/log_class.py:32        def get_content(self) -> list:
cloudra/base/log_class.py:33            '''
cloudra/base/log_class.py:34            获取日志内容，返回一个列表。
cloudra/base/log_class.py:35            '''
cloudra/base/log_class.py:36            if os.path.exists(self.path):
cloudra/base/log_class.py:37                with open(self.path, 'r', encoding='utf-8') as f:
cloudra/base/log_class.py:38                    content = f.readlines()
cloudra/base/log_class.py:39                    return content
cloudra/base/log_class.py:40            else:
cloudra/base/log_class.py:41                with open(self.path, 'w', encoding='utf-8') as f:
cloudra/base/log_class.py:42                    f.write('')
cloudra/base/log_class.py:43                    return []
cloudra/base/log_class.py:44
cloudra/base/log_class.py:45        def begin_stream(self) -> None:
cloudra/base/log_class.py:46            """流式输出开始时持有文件句柄，避免每个 token 都 open/write/close。"""
cloudra/base/log_class.py:47            if self._stream_file is None:
cloudra/base/log_class.py:48                self._stream_file = open(self.path, 'a', encoding='utf-8')
cloudra/base/log_class.py:49
cloudra/base/log_class.py:50        def end_stream(self) -> None:
cloudra/base/log_class.py:51            """流式输出结束，关闭句柄（close 会顺带 flush 缓冲）。"""
cloudra/base/log_class.py:52            if self._stream_file is not None:
cloudra/base/log_class.py:53                self._stream_file.close()
cloudra/base/log_class.py:54                self._stream_file = None
cloudra/base/log_class.py:55
cloudra/base/log_class.py:56        def update_content(self, new_content: str, print_to_console: bool = True) -> None:
cloudra/base/log_class.py:57            '''
cloudra/base/log_class.py:58            更新日志内容，追加新的内容。
cloudra/base/log_class.py:59            '''
cloudra/base/log_class.py:60            if print_to_console and self.is_main and Log.terminal_output:
cloudra/base/log_class.py:61                print(new_content, end='', flush=True)
cloudra/base/log_class.py:62
cloudra/base/log_class.py:63            # 流式期间复用已持有的句柄；非流式维持原有一次一开的写法
cloudra/base/log_class.py:64            if self._stream_file is not None:
cloudra/base/log_class.py:65                self._stream_file.write(new_content)
cloudra/base/log_class.py:66                return
cloudra/base/log_class.py:67            with open(self.path, 'a', encoding='utf-8') as f:
cloudra/base/log_class.py:68                f.write(new_content)
cloudra/base/session_class.py:1
```

```
cloudra/base/session_class.py:2 import os
cloudra/base/session_class.py:3 from datetime import datetime
cloudra/base/session_class.py:4
cloudra/base/session_class.py:5 from cloudra.persistence.fs_utils import atomic_write_json
cloudra/base/session_class.py:6
cloudra/base/session_class.py:7
cloudra/base/session_class.py:8 class Session:
cloudra/base/session_class.py:9     """内存态会话镜像；数据库（session 表）是唯一持久化事实来源。
cloudra/base/session_class.py:10
cloudra/base/session_class.py:11        默认不再写 per-session JSON 快照文件；调试时可设置环境变量
cloudra/base/session_class.py:12        ``CLOUDRA_DEBUG_SESSION_JSON=1`` 恢复写文件行为（upload_json 仍可显式调用）。
cloudra/base/session_class.py:13        """
cloudra/base/session_class.py:14        id: str
cloudra/base/session_class.py:15        path: str
cloudra/base/session_class.py:16        create_time: datetime
cloudra/base/session_class.py:17        state: str = 'sleeping'
cloudra/base/session_class.py:18
cloudra/base/session_class.py:19        def __init__(self, id: str, path: str) -> None:
cloudra/base/session_class.py:20            self.create_time = datetime.now()
cloudra/base/session_class.py:21            self.id = id
cloudra/base/session_class.py:22            self.path = path
cloudra/base/session_class.py:23            self.state = 'sleeping'
cloudra/base/session_class.py:24            if os.environ.get("CLOUDRA_DEBUG_SESSION_JSON") == "1":
cloudra/base/session_class.py:25                dir_name = os.path.dirname(self.path)
cloudra/base/session_class.py:26                if dir_name:
cloudra/base/session_class.py:27                    os.makedirs(dir_name, exist_ok=True)
cloudra/base/session_class.py:28                self.upload_json()
cloudra/base/session_class.py:29
cloudra/base/session_class.py:30
cloudra/base/session_class.py:31        def to_json(self) -> dict:
cloudra/base/session_class.py:32            create_time = self.create_time
cloudra/base/session_class.py:33            if isinstance(create_time, datetime):
cloudra/base/session_class.py:34                create_time = create_time.isoformat()
cloudra/base/session_class.py:35            json_dict = {
cloudra/base/session_class.py:36                'id': self.id,
cloudra/base/session_class.py:37                'path': self.path,
cloudra/base/session_class.py:38                'create_time': create_time,
cloudra/base/session_class.py:39                'state': self.state,
cloudra/base/session_class.py:40            }
cloudra/base/session_class.py:41            return json_dict
cloudra/base/session_class.py:42
cloudra/base/session_class.py:43        def upload_json(self):
cloudra/base/session_class.py:44            atomic_write_json(self.path, self.to_json())
cloudra/base/session_class.py:45
cloudra/base/skill_class.py:1   import hashlib
cloudra/base/skill_class.py:2   import os
cloudra/base/skill_class.py:3
cloudra/base/skill_class.py:4   import frontmatter as frt
cloudra/base/skill_class.py:5
cloudra/base/skill_class.py:6   # 模块级缓存按 SKILL.md 内容 revision 失效。Phase 6 后 skill manifest
cloudra/base/skill_class.py:7   # 允许在进程运行期间更新，不能再假设目录在整个进程生命周期中静态不变。
cloudra/base/skill_class.py:8   _SKILL_DATAS_CACHE: dict[str, tuple[str, list[dict]]] = {}
```

```
cloudra/base/skill_class.py:9
cloudra/base/skill_class.py:10  # NOTE: Skills 模型仅作为 skill frontmatter 的字段参照，当前未被实例化使用。
cloudra/base/skill_class.py:11  # 若后续需要校验 skill 元数据，可取消注释并补全默认值。
cloudra/base/skill_class.py:12  # class Skills(BaseModel):
cloudra/base/skill_class.py:13  #     name: str
cloudra/base/skill_class.py:14  #     description: str
cloudra/base/skill_class.py:15  #     path: str
cloudra/base/skill_class.py:16  #     scripts: list = []
cloudra/base/skill_class.py:17  #     references: list = []
cloudra/base/skill_class.py:18
cloudra/base/skill_class.py:19
cloudra/base/skill_class.py:20  class skill_loader:
cloudra/base/skill_class.py:21
cloudra/base/skill_class.py:22      # Skills are bundled with the package, not user data（base/ 上溯一级到包根）
cloudra/base/skill_class.py:23      base_path: str = os.path.join(
cloudra/base/skill_class.py:24          os.path.dirname(os.path.dirname(os.path.abspath(__file__))), 'skills'
cloudra/base/skill_class.py:25      )
cloudra/base/skill_class.py:26
cloudra/base/skill_class.py:27      def __init__(self):
cloudra/base/skill_class.py:28          # FIX: 原代码直接用了模块级变量 base_path（缺少 self.），在实例方法中访问类属性应加 self
cloudra/base/skill_class.py:29          self.namelst = os.listdir(self.base_path)
cloudra/base/skill_class.py:30
cloudra/base/skill_class.py:31          self.pathlst = [os.path.join(self.base_path, skill_name) for skill_name in self.
namelst]
cloudra/base/skill_class.py:32
cloudra/base/skill_class.py:33          self.md_pathlst = [os.path.join(path,'SKILL.md') for path in self.pathlst if os.
path.isdir(path)]
cloudra/base/skill_class.py:34
cloudra/base/skill_class.py:35      def skill_datas_get(self) -> list[dict]:
cloudra/base/skill_class.py:36          revision_hash = hashlib.sha256()
cloudra/base/skill_class.py:37          for md in sorted(self.md_pathlst):
cloudra/base/skill_class.py:38              revision_hash.update(md.encode("utf-8"))
cloudra/base/skill_class.py:39              try:
cloudra/base/skill_class.py:40                  with open(md, "rb") as stream:
cloudra/base/skill_class.py:41                      revision_hash.update(stream.read())
cloudra/base/skill_class.py:42              except OSError:
cloudra/base/skill_class.py:43                  continue
cloudra/base/skill_class.py:44          revision = revision_hash.hexdigest()
cloudra/base/skill_class.py:45          cached = _SKILL_DATAS_CACHE.get(self.base_path)
cloudra/base/skill_class.py:46          if cached is None or cached[0] != revision:
cloudra/base/skill_class.py:47              mds = self.md_pathlst
cloudra/base/skill_class.py:48              datas = []
cloudra/base/skill_class.py:49              for md in mds:
cloudra/base/skill_class.py:50                  with open(md, 'r', encoding='utf-8') as f:
cloudra/base/skill_class.py:51                      post = frt.load(f)
cloudra/base/skill_class.py:52                      metadata = post.metadata
cloudra/base/skill_class.py:53                      content = post.content
cloudra/base/skill_class.py:54
cloudra/base/skill_class.py:55                      # FIX: dict.update() 返回 None，原代码把 None 当数据 append 了
cloudra/base/skill_class.py:56                      # 这里直接修改 metadata dict 后 append（frontmatter 的 metadata 是普通 dict
)
cloudra/base/skill_class.py:57                      metadata['content'] = content
cloudra/base/skill_class.py:58                      datas.append(metadata)
cloudra/base/skill_class.py:59
cloudra/base/skill_class.py:60              _SKILL_DATAS_CACHE[self.base_path] = (revision, datas)
```

```
cloudra/base/skill_class.py:61
cloudra/base/skill_class.py:62          return _SKILL_DATAS_CACHE[self.base_path][1]
cloudra/base/skill_class.py:63
cloudra/base/skill_class.py:64      def get_reference(self, skill_name, ref_name) -> str:
cloudra/base/skill_class.py:65          # 支持多种可能的资料文件夹命名约定
cloudra/base/skill_class.py:66          possible_dirs = ['references', 'reference', 'ref']
cloudra/base/skill_class.py:67
cloudra/base/skill_class.py:68          for dir_name in possible_dirs:
cloudra/base/skill_class.py:69              ref_path = os.path.join(self.base_path, skill_name, dir_name, ref_name)
cloudra/base/skill_class.py:70              if os.path.isfile(ref_path):
cloudra/base/skill_class.py:71                  with open(ref_path, 'r', encoding='utf-8') as f:
cloudra/base/skill_class.py:72                      return f.read()
cloudra/base/skill_class.py:73              elif os.path.isdir(ref_path):
cloudra/base/skill_class.py:74                  dir_filelst = os.listdir(ref_path)
cloudra/base/skill_class.py:75                  return fr'该reference为文件夹，文件夹下有：{dir_filelst} ,请根据需要输入 {ref_name
}/filename 调用资料，win系统请用 {ref_name}\filename'
cloudra/base/skill_class.py:76
cloudra/base/skill_class.py:77          return '该资料不存在。'
cloudra/base/skill_class.py:78
cloudra/base/skill_class.py:79      # FIX: 原拼写为 skill_dsatas_get（多打了个 s）
cloudra/base/skill_class.py:80      def get_content(self, skill_name) -> str:
cloudra/base/skill_class.py:81          datas = self.skill_datas_get()
cloudra/base/skill_class.py:82          for data in datas:
cloudra/base/skill_class.py:83              if data['name'] == skill_name:
cloudra/base/skill_class.py:84                  return data['content']
cloudra/base/skill_class.py:85          return 'skill不存在。'
cloudra/base/skill_class.py:86
cloudra/base/skill_class.py:87
cloudra/base/skill_class.py:88
cloudra/base/skill_class.py:89
cloudra/base/skill_class.py:90
cloudra/base/skill_class.py:91
cloudra/base/skill_class.py:92
cloudra/base/skill_class.py:93
cloudra/base/skill_class.py:94
cloudra/base/skill_class.py:95
cloudra/base/skill_class.py:96
cloudra/base/skill_class.py:97
cloudra/base/skill_class.py:98
cloudra/base/skill_class.py:99
cloudra/base/tools_class.py:1   from contextlib import contextmanager
cloudra/base/tools_class.py:2   from contextvars import ContextVar
cloudra/base/tools_class.py:3   from pathlib import Path
cloudra/base/tools_class.py:4   from typing import TYPE_CHECKING, Any, Awaitable, Callable, Optional
cloudra/base/tools_class.py:5
cloudra/base/tools_class.py:6   from pydantic import BaseModel
cloudra/base/tools_class.py:7   from pydantic_core import PydanticUndefined
cloudra/base/tools_class.py:8
cloudra/base/tools_class.py:9   from cloudra.base.session_class import Session
cloudra/base/tools_class.py:10  from cloudra.persistence.database import ConversationDB, get_database_path
cloudra/base/tools_class.py:11
cloudra/base/tools_class.py:12  # HACK: 这里原本缺少 import，属于历史遗留问题；get_tools_lst 定义在 tools/__init__.py
cloudra/base/tools_class.py:13  from cloudra.tools import get_tools_lst
```

```
cloudra/base/tools_class.py:14  from cloudra.tools.core.permission_model import Permission
cloudra/base/tools_class.py:15
cloudra/base/tools_class.py:16  if TYPE_CHECKING:
cloudra/base/tools_class.py:17      from cloudra.infra.yolo_guard import YoloGuard
cloudra/base/tools_class.py:18
cloudra/base/tools_class.py:19  # runtime_v2 的审批钩子：LegacyToolExecutor 在执行工具前设置；未设置时
cloudra/base/tools_class.py:20  # （CLI/Flet/旧服务层）request_approval 维持原有的 SQLite + 事件总线等待。
cloudra/base/tools_class.py:21  # 签名：async (**fields) -> tuple[bool, str]，字段同 request_approval 的关键字参数。
cloudra/base/tools_class.py:22  _APPROVAL_HANDLER: ContextVar[
cloudra/base/tools_class.py:23      Optional[Callable[..., Awaitable[tuple[bool, str]]]]
cloudra/base/tools_class.py:24  ] = ContextVar("cloudra_approval_handler", default=None)
cloudra/base/tools_class.py:25
cloudra/base/tools_class.py:26
cloudra/base/tools_class.py:27  @contextmanager
cloudra/base/tools_class.py:28  def use_approval_handler(handler: Callable[..., Awaitable[tuple[bool, str]]]):
cloudra/base/tools_class.py:29      """Scope a v2 approval handler to one tool execution (per-task isolation)."""
cloudra/base/tools_class.py:30      token = _APPROVAL_HANDLER.set(handler)
cloudra/base/tools_class.py:31      try:
cloudra/base/tools_class.py:32          yield
cloudra/base/tools_class.py:33      finally:
cloudra/base/tools_class.py:34          _APPROVAL_HANDLER.reset(token)
cloudra/base/tools_class.py:35
cloudra/base/tools_class.py:36
cloudra/base/tools_class.py:37  def _tool_db_path(subagent) -> str:
cloudra/base/tools_class.py:38      """根据 tool 关联的 Agent 返回对应 conversation 的数据库路径。"""
cloudra/base/tools_class.py:39      conv_id = getattr(subagent, 'conversation_id', 'default') if subagent else 'default'
cloudra/base/tools_class.py:40      return get_database_path(conv_id)
cloudra/base/tools_class.py:41
cloudra/base/tools_class.py:42
cloudra/base/tools_class.py:43  def get_tools() -> list[dict]:
cloudra/base/tools_class.py:44      """将工具类加载为 OpenAI function 格式的 dict 列表。"""
cloudra/base/tools_class.py:45      entries = get_tools_lst()
cloudra/base/tools_class.py:46      tools_list = []
cloudra/base/tools_class.py:47      for entry in entries:
cloudra/base/tools_class.py:48          tool_instance = entry.instantiate()
cloudra/base/tools_class.py:49          tools_list.append(tool_instance.tool_schema_json())
cloudra/base/tools_class.py:50      return tools_list
cloudra/base/tools_class.py:51
cloudra/base/tools_class.py:52
cloudra/base/tools_class.py:53  class input_schema(BaseModel):
cloudra/base/tools_class.py:54      '''
cloudra/base/tools_class.py:55      工具输入模式基类，定义工具需要的输入参数和格式。
cloudra/base/tools_class.py:56
cloudra/base/tools_class.py:57      子类用法示例：
cloudra/base/tools_class.py:58          class MyInput(input_schema):
cloudra/base/tools_class.py:59              query: str = Field(description="搜索关键词")
cloudra/base/tools_class.py:60              max_results: int = Field(default=10, description="最大返回条数")
cloudra/base/tools_class.py:61
cloudra/base/tools_class.py:62      通过 to_parameters_schema() 可自动生成符合 OpenAI Function Calling 规范的 parameters 字典。
cloudra/base/tools_class.py:63      '''
cloudra/base/tools_class.py:64
cloudra/base/tools_class.py:65      @classmethod
```

```
cloudra/base/tools_class.py:66      def to_parameters_schema(cls) -> dict:
cloudra/base/tools_class.py:67          '''
cloudra/base/tools_class.py:68          生成符合 OpenAI Function Calling 的 parameters 字典。
cloudra/base/tools_class.py:69          基于 Pydantic model_json_schema() 微调：
cloudra/base/tools_class.py:70          - properties / required 已由 Pydantic 根据字段声明自动生成
cloudra/base/tools_class.py:71          - 强制附加 additionalProperties: false，防止 LLM 传入未定义字段
cloudra/base/tools_class.py:72          '''
cloudra/base/tools_class.py:73          schema = cls.model_json_schema()
cloudra/base/tools_class.py:74          parameters = {
cloudra/base/tools_class.py:75              "type": "object",
cloudra/base/tools_class.py:76              "properties": schema.get("properties", {}),
cloudra/base/tools_class.py:77              "required": schema.get("required", []),
cloudra/base/tools_class.py:78              "additionalProperties": False,
cloudra/base/tools_class.py:79          }
cloudra/base/tools_class.py:80          if "$defs" in schema:
cloudra/base/tools_class.py:81              parameters["$defs"] = schema["$defs"]
cloudra/base/tools_class.py:82          return parameters
cloudra/base/tools_class.py:83
cloudra/base/tools_class.py:84      @classmethod
cloudra/base/tools_class.py:85      def from_args(cls, arg_dict: dict) -> "input_schema":
cloudra/base/tools_class.py:86          '''从 LLM 返回的 arguments dict 构造输入实例，带 Pydantic 类型校验。'''
cloudra/base/tools_class.py:87          return cls(**arg_dict)
cloudra/base/tools_class.py:88
cloudra/base/tools_class.py:89      def toClass(self, arg_dict: dict) -> "input_schema":
cloudra/base/tools_class.py:90          '''
cloudra/base/tools_class.py:91          兼容旧接口，供 reAct.py 调用。
cloudra/base/tools_class.py:92          实际调用链通常为 tool.input_schema.toClass(tool_args)，此时 self 为 input_schema 子类本
身。
cloudra/base/tools_class.py:93          '''
cloudra/base/tools_class.py:94          # 当通过类调用时（如 TextReader_input_schema.toClass(args)），self 即为类对象
cloudra/base/tools_class.py:95          if isinstance(self, type) and issubclass(self, input_schema):
cloudra/base/tools_class.py:96              return self(**arg_dict)
cloudra/base/tools_class.py:97          return self.__class__(**arg_dict)
cloudra/base/tools_class.py:98
cloudra/base/tools_class.py:99      def toJson(self) -> dict:
cloudra/base/tools_class.py:100         '''实例序列化为 dict，用于日志或上下文回传。'''
cloudra/base/tools_class.py:101         return self.model_dump(mode="json")
cloudra/base/tools_class.py:102
cloudra/base/tools_class.py:103
cloudra/base/tools_class.py:104 class output_schema(BaseModel):
cloudra/base/tools_class.py:105     '''
cloudra/base/tools_class.py:106     工具输出模式基类，定义工具输出的格式。
cloudra/base/tools_class.py:107     子类按普通 Pydantic Model 声明字段即可。
cloudra/base/tools_class.py:108     '''
cloudra/base/tools_class.py:109
cloudra/base/tools_class.py:110     def toJson(self) -> dict:
cloudra/base/tools_class.py:111         return self.model_dump(mode="json")
cloudra/base/tools_class.py:112
cloudra/base/tools_class.py:113
cloudra/base/tools_class.py:114 class Tool(BaseModel):
cloudra/base/tools_class.py:115     '''
cloudra/base/tools_class.py:116     工具基类。所有具体工具应继承此类。
cloudra/base/tools_class.py:117
```

```
118 cloudra/base/tools_class.py:118     关于 input_schema / output_schema：
119 cloudra/base/tools_class.py:119     - 这里存放的是具体的 schema 类（如 TextReader_input_schema），而非实例。
120 cloudra/base/tools_class.py:120     - 使用 Any 类型并开启 arbitrary_types_allowed，避免 Pydantic 对类对象做字段校验。
121 cloudra/base/tools_class.py:121     - 子模块中覆盖时必须带类型注解，例如：
122 cloudra/base/tools_class.py:122           input_schema: Any = TextReader_input_schema
123 cloudra/base/tools_class.py:123     '''
124 cloudra/base/tools_class.py:124     model_config = {"arbitrary_types_allowed": True}
125 cloudra/base/tools_class.py:125
126 cloudra/base/tools_class.py:126     # 唯一标识符
127 cloudra/base/tools_class.py:127     tool_id: str = ''
128 cloudra/base/tools_class.py:128
129 cloudra/base/tools_class.py:129     # 工具名称（即 OpenAI function.name）
130 cloudra/base/tools_class.py:130     tool_name: str = ''
131 cloudra/base/tools_class.py:131
132 cloudra/base/tools_class.py:132     # 工具描述。改造后应仅保留【功能描述】+【使用场景】，不再包含参数细节。
133 cloudra/base/tools_class.py:133     # 参数说明通过 input_schema 的 Field(description="...") 提供，自动生成到 parameters 中。
134 cloudra/base/tools_class.py:134     tool_description: str = ''
135 cloudra/base/tools_class.py:135
136 cloudra/base/tools_class.py:136     # 工具的输入模式类（存放类对象，非实例）
137 cloudra/base/tools_class.py:137     input_schema: Any = input_schema
138 cloudra/base/tools_class.py:138
139 cloudra/base/tools_class.py:139     # 工具的输出模式类（存放类对象，非实例）
140 cloudra/base/tools_class.py:140     output_schema: Any = output_schema
141 cloudra/base/tools_class.py:141
142 cloudra/base/tools_class.py:142     # 引用调用该工具的 Agent 实例
143 cloudra/base/tools_class.py:143     subagent: Any = None
144 cloudra/base/tools_class.py:144
145 cloudra/base/tools_class.py:145     def is_read(self) -> bool:
146 cloudra/base/tools_class.py:146         return False
147 cloudra/base/tools_class.py:147
148 cloudra/base/tools_class.py:148     def is_write(self) -> bool:
149 cloudra/base/tools_class.py:149         return False
150 cloudra/base/tools_class.py:150
151 cloudra/base/tools_class.py:151     def is_manage(self) -> bool:
152 cloudra/base/tools_class.py:152         return False
153 cloudra/base/tools_class.py:153
154 cloudra/base/tools_class.py:154     def is_script(self) -> bool:
155 cloudra/base/tools_class.py:155         return False
156 cloudra/base/tools_class.py:156
157 cloudra/base/tools_class.py:157     def is_destructive(self) -> bool:
158 cloudra/base/tools_class.py:158         return False
159 cloudra/base/tools_class.py:159
160 cloudra/base/tools_class.py:160     def tool_permission_all(self) -> Permission:
161 cloudra/base/tools_class.py:161         """Return Permission flags for this tool."""
162 cloudra/base/tools_class.py:162         perms = Permission.NONE
163 cloudra/base/tools_class.py:163         if self.is_read():
164 cloudra/base/tools_class.py:164             perms |= Permission.READ
165 cloudra/base/tools_class.py:165         if self.is_write():
166 cloudra/base/tools_class.py:166             perms |= Permission.WRITE
167 cloudra/base/tools_class.py:167         if self.is_manage():
168 cloudra/base/tools_class.py:168             perms |= Permission.MANAGE
169 cloudra/base/tools_class.py:169         if self.is_script():
```

```
cloudra/base/tools_class.py:170             perms |= Permission.SCRIPT
cloudra/base/tools_class.py:171         if self.is_destructive():
cloudra/base/tools_class.py:172             perms |= Permission.DESTRUCTIVE
cloudra/base/tools_class.py:173         return perms
cloudra/base/tools_class.py:174
cloudra/base/tools_class.py:175     async def tool_call(self, user_input: input_schema, session: Session) -> output_schema:
cloudra/base/tools_class.py:176         '''
cloudra/base/tools_class.py:177         调用工具。子类必须重写此方法。
cloudra/base/tools_class.py:178         执行结果以 output_schema 实例返回。
cloudra/base/tools_class.py:179         '''
cloudra/base/tools_class.py:180         raise NotImplementedError("子类必须实现 tool_call 方法")
cloudra/base/tools_class.py:181
cloudra/base/tools_class.py:182     async def safe_call(self, user_input: input_schema, session: Session) -> output_schema:
cloudra/base/tools_class.py:183         '''
cloudra/base/tools_class.py:184         带有通用异常捕获的工具调用包装器。
cloudra/base/tools_class.py:185         工具执行失败时抛出包含工具名的 RuntimeError，由 reAct.act_result() 统一捕获并回传给 LLM。
cloudra/base/tools_class.py:186         消息以 [Tool error] 前缀 + 工具名 + 恢复指引成形；react 层识别该前缀后不再二次套壳。
cloudra/base/tools_class.py:187         '''
cloudra/base/tools_class.py:188         try:
cloudra/base/tools_class.py:189             return await self.tool_call(user_input, session)
cloudra/base/tools_class.py:190         except Exception as e:
cloudra/base/tools_class.py:191             error_msg = (
cloudra/base/tools_class.py:192                 f"[Tool error] {self.tool_name}: {str(e)}. "
cloudra/base/tools_class.py:193                 "Fix the arguments or use a different tool, then retry."
cloudra/base/tools_class.py:194             )
cloudra/base/tools_class.py:195             raise RuntimeError(error_msg) from e
cloudra/base/tools_class.py:196
cloudra/base/tools_class.py:197     async def ensure_safety(
cloudra/base/tools_class.py:198         self,
cloudra/base/tools_class.py:199         session: Session,
cloudra/base/tools_class.py:200         details: str,
cloudra/base/tools_class.py:201         tool_args: dict | None = None,
cloudra/base/tools_class.py:202     ) -> tuple[bool, str]:
cloudra/base/tools_class.py:203         '''
cloudra/base/tools_class.py:204         安全检查：如果工具被标记为 destructive，则必须经过审批。
cloudra/base/tools_class.py:205         默认通过数据库等待用户审批；若启用了 YOLO 模式，则调用独立 Agent 自动判断。
cloudra/base/tools_class.py:206         返回 (是否通过, 拒绝原因)。通过时拒绝原因为空字符串。
cloudra/base/tools_class.py:207         '''
cloudra/base/tools_class.py:208         from cloudra.infra.yolo_guard import YoloGuard, is_yolo_enabled
cloudra/base/tools_class.py:209
cloudra/base/tools_class.py:210         # 获取当前 Agent 名称
cloudra/base/tools_class.py:211         agent_name = self.subagent.name if self.subagent else "Unknown Agent"
cloudra/base/tools_class.py:212         full_details = f"【发起 Agent】: {agent_name}\n{details}"
cloudra/base/tools_class.py:213
cloudra/base/tools_class.py:214         print(f"\n\033[91m[安全审计]\033[0m 正在申请危险操作审批...\n{full_details}")
cloudra/base/tools_class.py:215
cloudra/base/tools_class.py:216         if is_yolo_enabled():
cloudra/base/tools_class.py:217             guard = YoloGuard()
cloudra/base/tools_class.py:218             return await self._yolo_judge(session, tool_args or {}, guard)
cloudra/base/tools_class.py:219
cloudra/base/tools_class.py:220         db_path = _tool_db_path(self.subagent)
cloudra/base/tools_class.py:221         risk_summary = next(
```

```
cloudra/base/tools_class.py:222             (
cloudra/base/tools_class.py:223                 line.partition(":")[2].strip()
cloudra/base/tools_class.py:224                 for line in details.splitlines()
cloudra/base/tools_class.py:225                 if line.startswith("【动作】")
cloudra/base/tools_class.py:226             ),
cloudra/base/tools_class.py:227             details.splitlines()[0] if details.splitlines() else "危险工具调用",
cloudra/base/tools_class.py:228         )
cloudra/base/tools_class.py:229         candidate_diff = ""
cloudra/base/tools_class.py:230         diff_marker = "【Unified Diff】:\n"
cloudra/base/tools_class.py:231         if diff_marker in details:
cloudra/base/tools_class.py:232             candidate_diff = details.partition(diff_marker)[2]
cloudra/base/tools_class.py:233         return await self.request_approval(
cloudra/base/tools_class.py:234             db_path,
cloudra/base/tools_class.py:235             session.id,
cloudra/base/tools_class.py:236             full_details,
cloudra/base/tools_class.py:237             tool_name=self.tool_name,
cloudra/base/tools_class.py:238             normalized_arguments=tool_args or {},
cloudra/base/tools_class.py:239             risk_summary=risk_summary,
cloudra/base/tools_class.py:240             candidate_diff=candidate_diff,
cloudra/base/tools_class.py:241         )
cloudra/base/tools_class.py:242
cloudra/base/tools_class.py:243     async def record_session_event(
cloudra/base/tools_class.py:244         self,
cloudra/base/tools_class.py:245         session: Session,
cloudra/base/tools_class.py:246         event_type: str,
cloudra/base/tools_class.py:247         data: dict,
cloudra/base/tools_class.py:248     ) -> None:
cloudra/base/tools_class.py:249         """尽力记录工具事件；遥测失败不能反向破坏工具主流程。"""
cloudra/base/tools_class.py:250         from cloudra.services import session_store as _ss
cloudra/base/tools_class.py:251
cloudra/base/tools_class.py:252         try:
cloudra/base/tools_class.py:253             db = ConversationDB.from_path(_tool_db_path(self.subagent))
cloudra/base/tools_class.py:254             await _ss.arecord_event(db, session.id, event_type, data)
cloudra/base/tools_class.py:255         except Exception:
cloudra/base/tools_class.py:256             pass
cloudra/base/tools_class.py:257
cloudra/base/tools_class.py:258     async def _yolo_judge(
cloudra/base/tools_class.py:259         self,
cloudra/base/tools_class.py:260         session: Session,
cloudra/base/tools_class.py:261         tool_args: dict,
cloudra/base/tools_class.py:262         guard: 'YoloGuard',
cloudra/base/tools_class.py:263     ) -> tuple[bool, str]:
cloudra/base/tools_class.py:264         """调用 YoloGuard 做自动判断。"""
cloudra/base/tools_class.py:265         approved, reason = await guard.judge(
cloudra/base/tools_class.py:266             self.tool_name, tool_args, self.subagent, session
cloudra/base/tools_class.py:267         )
cloudra/base/tools_class.py:268         if approved:
cloudra/base/tools_class.py:269             print("\033[92m[安全审计/YOLO]\033[0m 自动审批通过。")
cloudra/base/tools_class.py:270             return True, ""
cloudra/base/tools_class.py:271         else:
cloudra/base/tools_class.py:272             reason_text = reason or "YOLO Agent 认为该操作存在风险"
cloudra/base/tools_class.py:273             print(f"\033[91m[安全审计/YOLO]\033[0m 自动审批拒绝。原因: {reason_text}")
```

```
cloudra/base/tools_class.py:274             return False, reason_text
cloudra/base/tools_class.py:275
cloudra/base/tools_class.py:276     async def request_approval(
cloudra/base/tools_class.py:277         self,
cloudra/base/tools_class.py:278         db_path: str,
cloudra/base/tools_class.py:279         session_id: str,
cloudra/base/tools_class.py:280         details: str,
cloudra/base/tools_class.py:281         *,
cloudra/base/tools_class.py:282         tool_name: str = "",
cloudra/base/tools_class.py:283         normalized_arguments: dict | None = None,
cloudra/base/tools_class.py:284         risk_summary: str = "",
cloudra/base/tools_class.py:285         candidate_diff: str = "",
cloudra/base/tools_class.py:286     ) -> tuple[bool, str]:
cloudra/base/tools_class.py:287         import asyncio
cloudra/base/tools_class.py:288
cloudra/base/tools_class.py:289         from cloudra.infra import events
cloudra/base/tools_class.py:290         from cloudra.services import session_store as _ss
cloudra/base/tools_class.py:291
cloudra/base/tools_class.py:292         handler = _APPROVAL_HANDLER.get()
cloudra/base/tools_class.py:293         if handler is not None:
cloudra/base/tools_class.py:294             # runtime_v2 上下文：审批请求与等待由内核 broker 承担，
cloudra/base/tools_class.py:295             # 不碰 conversation SQLite 与 legacy 事件总线。
cloudra/base/tools_class.py:296             return await handler(
cloudra/base/tools_class.py:297                 details=details,
cloudra/base/tools_class.py:298                 tool_name=tool_name,
cloudra/base/tools_class.py:299                 normalized_arguments=normalized_arguments or {},
cloudra/base/tools_class.py:300                 risk_summary=risk_summary,
cloudra/base/tools_class.py:301                 candidate_diff=candidate_diff,
cloudra/base/tools_class.py:302             )
cloudra/base/tools_class.py:303
cloudra/base/tools_class.py:304         db = ConversationDB.from_path(db_path)
cloudra/base/tools_class.py:305         conversation_id = getattr(self.subagent, "conversation_id", None)
cloudra/base/tools_class.py:306         if not conversation_id:
cloudra/base/tools_class.py:307             conversation_id = Path(db_path).parent.parent.name
cloudra/base/tools_class.py:308         decisions = await events.subscribe_async(conversation_id, "approval_decided")
cloudra/base/tools_class.py:309         approval_id = await _ss.ainsert_approval(
cloudra/base/tools_class.py:310             db,
cloudra/base/tools_class.py:311             session_id,
cloudra/base/tools_class.py:312             details,
cloudra/base/tools_class.py:313             agent_name=(
cloudra/base/tools_class.py:314                 self.subagent.name if self.subagent else "Unknown Agent"
cloudra/base/tools_class.py:315             ),
cloudra/base/tools_class.py:316             tool_name=tool_name,
cloudra/base/tools_class.py:317             normalized_arguments=normalized_arguments,
cloudra/base/tools_class.py:318             risk_summary=risk_summary,
cloudra/base/tools_class.py:319             candidate_diff=candidate_diff,
cloudra/base/tools_class.py:320         )
cloudra/base/tools_class.py:321         changed = await _ss.atransition_session_state(
cloudra/base/tools_class.py:322             db,
cloudra/base/tools_class.py:323             session_id,
cloudra/base/tools_class.py:324             expected={"running"},
cloudra/base/tools_class.py:325             target="waiting_for_approval",
```

```
cloudra/base/tools_class.py:326         )
cloudra/base/tools_class.py:327         if changed:
cloudra/base/tools_class.py:328             await _ss.ainsert_session_event(
cloudra/base/tools_class.py:329                 db,
cloudra/base/tools_class.py:330                 session_id,
cloudra/base/tools_class.py:331                 "waiting_for_approval",
cloudra/base/tools_class.py:332                 {
cloudra/base/tools_class.py:333                     "approval_id": approval_id,
cloudra/base/tools_class.py:334                     "state": "waiting_for_approval",
cloudra/base/tools_class.py:335                 },
cloudra/base/tools_class.py:336             )
cloudra/base/tools_class.py:337         try:
cloudra/base/tools_class.py:338             while True:
cloudra/base/tools_class.py:339                 try:
cloudra/base/tools_class.py:340                     message = await asyncio.wait_for(decisions.get(), timeout=30.0)
cloudra/base/tools_class.py:341                 except asyncio.TimeoutError:
cloudra/base/tools_class.py:342                     message = None
cloudra/base/tools_class.py:343                 if message and message.get("approval_id") == approval_id:
cloudra/base/tools_class.py:344                     state = message.get("state")
cloudra/base/tools_class.py:345                     denial_reason = message.get("denial_reason", "")
cloudra/base/tools_class.py:346                     if state == "approved":
cloudra/base/tools_class.py:347                         print("\033[92m[安全审计]\033[0m 审批通过。")
cloudra/base/tools_class.py:348                         return True, ""
cloudra/base/tools_class.py:349                     if state == "denied":
cloudra/base/tools_class.py:350                         return False, denial_reason
cloudra/base/tools_class.py:351                 row = await _ss.aread_approval_status(db, approval_id)
cloudra/base/tools_class.py:352                 if row:
cloudra/base/tools_class.py:353                     state, denial_reason = row
cloudra/base/tools_class.py:354                     if state == 'approved':
cloudra/base/tools_class.py:355                         print("\033[92m[安全审计]\033[0m 审批通过。")
cloudra/base/tools_class.py:356                         return True, ''
cloudra/base/tools_class.py:357                     elif state == 'denied':
cloudra/base/tools_class.py:358                         reason = denial_reason or ''
cloudra/base/tools_class.py:359                         if reason:
cloudra/base/tools_class.py:360                             print(f"\033[91m[安全审计]\033[0m 审批拒绝。原因: {reason}")
cloudra/base/tools_class.py:361                         else:
cloudra/base/tools_class.py:362                             print("\033[91m[安全审计]\033[0m 审批拒绝。")
cloudra/base/tools_class.py:363                         return False, reason
cloudra/base/tools_class.py:364                     elif state == "expired":
cloudra/base/tools_class.py:365                         print("\033[91m[安全审计]\033[0m 审批已过期。")
cloudra/base/tools_class.py:366                         return False, "审批已过期"
cloudra/base/tools_class.py:367         finally:
cloudra/base/tools_class.py:368             restored = await _ss.atransition_session_state(
cloudra/base/tools_class.py:369                 db,
cloudra/base/tools_class.py:370                 session_id,
cloudra/base/tools_class.py:371                 expected={"waiting_for_approval"},
cloudra/base/tools_class.py:372                 target="running",
cloudra/base/tools_class.py:373             )
cloudra/base/tools_class.py:374             if restored:
cloudra/base/tools_class.py:375                 await _ss.ainsert_session_event(
cloudra/base/tools_class.py:376                     db,
cloudra/base/tools_class.py:377                     session_id,
```

```
cloudra/base/tools_class.py:378                    "running",
cloudra/base/tools_class.py:379                    {
cloudra/base/tools_class.py:380                        "approval_id": approval_id,
cloudra/base/tools_class.py:381                        "state": "running",
cloudra/base/tools_class.py:382                    },
cloudra/base/tools_class.py:383                )
cloudra/base/tools_class.py:384
cloudra/base/tools_class.py:385        @classmethod
cloudra/base/tools_class.py:386        def _get_field_default(cls, field_name: str):
cloudra/base/tools_class.py:387            '''获取类字段的默认值，兼容 Pydantic v2（类级别无法直接访问普通字段）。'''
cloudra/base/tools_class.py:388            field = cls.model_fields.get(field_name)
cloudra/base/tools_class.py:389            if field and field.default is not PydanticUndefined:
cloudra/base/tools_class.py:390                return field.default
cloudra/base/tools_class.py:391            return None
cloudra/base/tools_class.py:392
cloudra/base/tools_class.py:393        def tool_schema_json(self) -> dict:
cloudra/base/tools_class.py:394            '''
cloudra/base/tools_class.py:395            返回标准 OpenAI function 格式，可直接用于 client.chat.completions.create(..., tools
=[...])。
cloudra/base/tools_class.py:396            优先读取实例级别的 tool_description，使动态生成的描述能够生效。
cloudra/base/tools_class.py:397            '''
cloudra/base/tools_class.py:398            # HACK: 这里用 getattr + or fallback 链是因为 Pydantic v2 的类字段默认值在实例上可能
cloudra/base/tools_class.py:399            # 被 shadow，但某些情况下实例属性又还没覆盖。动态 description（如 Bashtools 的权限列表、
cloudra/base/tools_class.py:400            # GetSkill 的 skills 列表）必须在 __init__ 之后才能确定，因此必须是实例方法而非类方法。
cloudra/base/tools_class.py:401            # 如果后续有更好方式（如 descriptor / property），可替换掉这个 ugly fallback。
cloudra/base/tools_class.py:402            description = getattr(self, 'tool_description', '') or self._get_field_default('
tool_description') or ''
cloudra/base/tools_class.py:403            return {
cloudra/base/tools_class.py:404                "type": "function",
cloudra/base/tools_class.py:405                "function": {
cloudra/base/tools_class.py:406                    "name": getattr(self, 'tool_name', '') or self._get_field_default('
tool_name') or '',
cloudra/base/tools_class.py:407                    "description": description,
cloudra/base/tools_class.py:408                    "parameters": (self._get_field_default('input_schema') or input_schema).
to_parameters_schema(),
cloudra/base/tools_class.py:409                }
cloudra/base/tools_class.py:410            }
cloudra/base/tools_class.py:411
cloudra/base/tools_class.py:412        def toJson(self) -> dict:
cloudra/base/tools_class.py:413            '''
cloudra/base/tools_class.py:414            实例级序列化，主要用于日志或调试。
cloudra/base/tools_class.py:415            若需获取 OpenAI schema，请使用类方法 tool_schema_json()。
cloudra/base/tools_class.py:416            '''
cloudra/base/tools_class.py:417            return {
cloudra/base/tools_class.py:418                "tool_id": self.tool_id,
cloudra/base/tools_class.py:419                "tool_name": self.tool_name,
cloudra/base/tools_class.py:420                "tool_description": self.tool_description,
cloudra/base/tools_class.py:421            }
cloudra/base/tools_class.py:422
cloudra/base/tools_class.py:423        @classmethod
cloudra/base/tools_class.py:424        def build_input(cls, arg_dict: dict) -> input_schema:
cloudra/base/tools_class.py:425            '''
cloudra/base/tools_class.py:426            将 LLM 返回的参数字典转换为 input_schema 实例。
cloudra/base/tools_class.py:427            供 reAct.py 调用，替代原有的 tool.input_schema.toClass(tool_args)。
cloudra/base/tools_class.py:428            '''
cloudra/base/tools_class.py:429            schema_cls = cls._get_field_default('input_schema') or input_schema
```

```
cloudra/base/tools_class.py:430         return schema_cls.from_args(arg_dict)
cloudra/cli.py:1    """
cloudra/cli.py:2    Cloudra CLI - 自主有机小分子机理计算 Agent 命令行工具。
cloudra/cli.py:3    """
cloudra/cli.py:4
cloudra/cli.py:5    import sys
cloudra/cli.py:6    from pathlib import Path
cloudra/cli.py:7
cloudra/cli.py:8    import click
cloudra/cli.py:9    from rich.console import Console
cloudra/cli.py:10   from rich.panel import Panel
cloudra/cli.py:11   from rich.table import Table
cloudra/cli.py:12
cloudra/cli.py:13   from cloudra import __version__
cloudra/cli.py:14   from cloudra.infra.config import get_config
cloudra/cli.py:15   from cloudra.persistence import paths
cloudra/cli.py:16   from cloudra.web_launcher import launch_browser
cloudra/cli.py:17
cloudra/cli.py:18   console = Console()
cloudra/cli.py:19   APP_NAME = "cloudra"
cloudra/cli.py:20   VERSION = __version__
cloudra/cli.py:21
cloudra/cli.py:22
cloudra/cli.py:23   @click.group(invoke_without_command=True)
cloudra/cli.py:24   @click.option("--version", is_flag=True, help="显示版本信息")
cloudra/cli.py:25   @click.option("--coordinator-model", envvar="CLOUDRA_COORDINATOR_MODEL", help="覆盖 Coordinator 模型")
cloudra/cli.py:26   @click.option("--coordinator-url", envvar="CLOUDRA_COORDINATOR_URL", help="覆盖 Coordinator API 地址")
cloudra/cli.py:27   @click.option("--coordinator-key", envvar="CLOUDRA_COORDINATOR_KEY", help="覆盖 Coordinator API Key")
cloudra/cli.py:28   @click.option("--follower-model", envvar="CLOUDRA_FOLLOWER_MODEL", help="覆盖 Follower 模型")
cloudra/cli.py:29   @click.option("--follower-url", envvar="CLOUDRA_FOLLOWER_URL", help="覆盖 Follower API 地址")
cloudra/cli.py:30   @click.option("--follower-key", envvar="CLOUDRA_FOLLOWER_KEY", help="覆盖 Follower API Key")
cloudra/cli.py:31   @click.pass_context
cloudra/cli.py:32   def cli(ctx, version, coordinator_model, coordinator_url, coordinator_key,
cloudra/cli.py:33           follower_model, follower_url, follower_key):
cloudra/cli.py:34       """Cloudra - 自主有机小分子机理计算 Agent"""
cloudra/cli.py:35       if version:
cloudra/cli.py:36           console.print(f"[bold cyan]Cloudra[/bold cyan] version {VERSION}")
cloudra/cli.py:37           sys.exit(0)
cloudra/cli.py:38
cloudra/cli.py:39       if ctx.invoked_subcommand is None:
cloudra/cli.py:40           _apply_config_overrides(
cloudra/cli.py:41               coordinator_model, coordinator_url, coordinator_key,
cloudra/cli.py:42               follower_model, follower_url, follower_key,
cloudra/cli.py:43           )
cloudra/cli.py:44           sys.exit(launch_browser(_frontend_or_exit(), rebuild=False, skip_build=False))
cloudra/cli.py:45
cloudra/cli.py:46       _apply_config_overrides(
cloudra/cli.py:47           coordinator_model, coordinator_url, coordinator_key,
cloudra/cli.py:48           follower_model, follower_url, follower_key,
cloudra/cli.py:49       )
cloudra/cli.py:50
cloudra/cli.py:51
```

```
cloudra/cli.py:52   def _apply_config_overrides(coordinator_model, coordinator_url, coordinator_key,
cloudra/cli.py:53                               follower_model, follower_url, follower_key):
cloudra/cli.py:54       """把 CLI 覆盖项写入全局配置。"""
cloudra/cli.py:55       config = get_config()
cloudra/cli.py:56       if coordinator_model:
cloudra/cli.py:57           config.coordinator.model = coordinator_model
cloudra/cli.py:58       if coordinator_url:
cloudra/cli.py:59           config.coordinator.url = coordinator_url
cloudra/cli.py:60       if coordinator_key:
cloudra/cli.py:61           config.coordinator.key = coordinator_key
cloudra/cli.py:62       if follower_model:
cloudra/cli.py:63           config.follower.model = follower_model
cloudra/cli.py:64       if follower_url:
cloudra/cli.py:65           config.follower.url = follower_url
cloudra/cli.py:66       if follower_key:
cloudra/cli.py:67           config.follower.key = follower_key
cloudra/cli.py:68
cloudra/cli.py:69
cloudra/cli.py:70   def _frontend_or_exit():
cloudra/cli.py:71       from cloudra.web_launcher import _frontend_dir
cloudra/cli.py:72       try:
cloudra/cli.py:73           return _frontend_dir()
cloudra/cli.py:74       except RuntimeError as exc:
cloudra/cli.py:75           console.print(f"[red]{exc}[/red]")
cloudra/cli.py:76           sys.exit(1)
cloudra/cli.py:77
cloudra/cli.py:78
cloudra/cli.py:79   @cli.command(name="web")
cloudra/cli.py:80   @click.option("--rebuild", is_flag=True, help="强制重新构建前端")
cloudra/cli.py:81   @click.option("--skip-build", is_flag=True, help="复用已有 frontend/dist 构建")
cloudra/cli.py:82   def web(rebuild, skip_build):
cloudra/cli.py:83       """启动 Web 前端（API + SPA 单端口服务）。"""
cloudra/cli.py:84       sys.exit(launch_browser(_frontend_or_exit(), rebuild=rebuild, skip_build=skip_build))
cloudra/cli.py:85
cloudra/cli.py:86
cloudra/cli.py:87   @cli.group(name="config")
cloudra/cli.py:88   def config_cmd():
cloudra/cli.py:89       """配置管理（init, show, get, set）"""
cloudra/cli.py:90       pass
cloudra/cli.py:91
cloudra/cli.py:92
cloudra/cli.py:93   @config_cmd.command(name="init")
cloudra/cli.py:94   @click.option("--force", is_flag=True, help="覆盖已有配置文件")
cloudra/cli.py:95   def config_init(force):
cloudra/cli.py:96       """交互式初始化配置文件。"""
cloudra/cli.py:97       cfg = get_config()
cloudra/cli.py:98       cfg_path = cfg.config_file
cloudra/cli.py:99
cloudra/cli.py:100      if cfg_path.exists() and not force:
cloudra/cli.py:101          console.print(f"[yellow]配置文件已存在: {cfg_path}[/yellow]")
cloudra/cli.py:102          console.print("使用 `--force` 覆盖，或直接编辑该文件。")
cloudra/cli.py:103          return
```

```
cloudra/cli.py:104
cloudra/cli.py:105      console.print("[bold]Cloudra 配置初始化[/bold]\n")
cloudra/cli.py:106
cloudra/cli.py:107      # Prompt for values (use existing as defaults)
cloudra/cli.py:108      coord_model = click.prompt("Coordinator 模型", default=cfg.coordinator.model)
cloudra/cli.py:109      coord_url = click.prompt("Coordinator API 地址", default=cfg.coordinator.url)
cloudra/cli.py:110      coord_key = click.prompt("Coordinator API Key", default="", hide_input=True)
cloudra/cli.py:111
cloudra/cli.py:112      follower_model = click.prompt("Follower 模型", default=cfg.follower.model)
cloudra/cli.py:113      follower_url = click.prompt("Follower API 地址", default=cfg.follower.url)
cloudra/cli.py:114      follower_key = click.prompt("Follower API Key", default="", hide_input=True)
cloudra/cli.py:115      openalex_key = click.prompt(
cloudra/cli.py:116          "OpenAlex API Key（可选，回车跳过；无 key 时匿名检索）",
cloudra/cli.py:117          default="",
cloudra/cli.py:118          hide_input=True,
cloudra/cli.py:119      )
cloudra/cli.py:120      openalex_mailto = click.prompt(
cloudra/cli.py:121          "OpenAlex 礼貌池邮箱（可选，回车跳过）",
cloudra/cli.py:122          default=cfg.openalex_mailto or "",
cloudra/cli.py:123      )
cloudra/cli.py:124
cloudra/cli.py:125      cfg.coordinator.model = coord_model
cloudra/cli.py:126      cfg.coordinator.url = coord_url
cloudra/cli.py:127      cfg.coordinator.key = coord_key or cfg.coordinator.key
cloudra/cli.py:128      cfg.follower.model = follower_model
cloudra/cli.py:129      cfg.follower.url = follower_url
cloudra/cli.py:130      cfg.follower.key = follower_key or cfg.follower.key
cloudra/cli.py:131      if openalex_key:
cloudra/cli.py:132          cfg.openalex_api_key = openalex_key
cloudra/cli.py:133      if openalex_mailto:
cloudra/cli.py:134          cfg.openalex_mailto = openalex_mailto
cloudra/cli.py:135
cloudra/cli.py:136      cfg.save()
cloudra/cli.py:137      console.print(f"[green]配置已保存至: {cfg_path}[/green]")
cloudra/cli.py:138
cloudra/cli.py:139
cloudra/cli.py:140  @config_cmd.command(name="show")
cloudra/cli.py:141  def config_show():
cloudra/cli.py:142      """显示当前生效的配置（敏感信息脱敏）。"""
cloudra/cli.py:143      cfg = get_config()
cloudra/cli.py:144      display = cfg.to_display_dict()
cloudra/cli.py:145
cloudra/cli.py:146      console.print(Panel("[bold]Cloudra 配置[/bold]", style="cyan"))
cloudra/cli.py:147      console.print(f"[bold]Data Directory:[/bold] {display['data_dir']}")
cloudra/cli.py:148      console.print(f"[bold]Config File:[/bold] {display['config_file']}")
cloudra/cli.py:149      console.print()
cloudra/cli.py:150
cloudra/cli.py:151      table = Table(title="LLM 配置", show_header=True, header_style="bold magenta")
cloudra/cli.py:152      table.add_column("角色")
cloudra/cli.py:153      table.add_column("模型")
cloudra/cli.py:154      table.add_column("URL")
cloudra/cli.py:155      table.add_column("Key")
```

```
cloudra/cli.py:156
cloudra/cli.py:157      table.add_row(
cloudra/cli.py:158          "Coordinator",
cloudra/cli.py:159          display['llm']['coordinator']['model'],
cloudra/cli.py:160          display['llm']['coordinator']['url'],
cloudra/cli.py:161          display['llm']['coordinator']['key'],
cloudra/cli.py:162      )
cloudra/cli.py:163      table.add_row(
cloudra/cli.py:164          "Follower",
cloudra/cli.py:165          display['llm']['follower']['model'],
cloudra/cli.py:166          display['llm']['follower']['url'],
cloudra/cli.py:167          display['llm']['follower']['key'],
cloudra/cli.py:168      )
cloudra/cli.py:169
cloudra/cli.py:170      console.print(table)
cloudra/cli.py:171      console.print()
cloudra/cli.py:172      oa_key = display.get("openalex_api_key", "(not set)")
cloudra/cli.py:173      oa_mail = display.get("openalex_mailto", "(not set)")
cloudra/cli.py:174      console.print(f"[bold]OpenAlex API Key:[/bold] {oa_key}")
cloudra/cli.py:175      console.print(f"[bold]OpenAlex mailto:[/bold] {oa_mail}")
cloudra/cli.py:176
cloudra/cli.py:177
cloudra/cli.py:178  def _resolve_config_attr(cfg, key: str):
cloudra/cli.py:179      """Backward-compatible alias; 实现已下沉到 cloudra.infra.config 供 API 复用。"""
cloudra/cli.py:180      from cloudra.infra.config import resolve_config_attr
cloudra/cli.py:181      return resolve_config_attr(cfg, key)
cloudra/cli.py:182
cloudra/cli.py:183
cloudra/cli.py:184  @config_cmd.command(name="get")
cloudra/cli.py:185  @click.argument("key")
cloudra/cli.py:186  def config_get(key):
cloudra/cli.py:187      """读取单个配置项。例如: cloudra config get coordinator.model"""
cloudra/cli.py:188      cfg = get_config()
cloudra/cli.py:189
cloudra/cli.py:190      target, final_key = _resolve_config_attr(cfg, key)
cloudra/cli.py:191      if target is None:
cloudra/cli.py:192          # Try without 'llm.' prefix for convenience
cloudra/cli.py:193          target, final_key = _resolve_config_attr(cfg, f"coordinator.{key}")
cloudra/cli.py:194      if target is None:
cloudra/cli.py:195          target, final_key = _resolve_config_attr(cfg, f"follower.{key}")
cloudra/cli.py:196      if target is None:
cloudra/cli.py:197          console.print(f"[red]配置项不存在: {key}[/red]")
cloudra/cli.py:198          sys.exit(1)
cloudra/cli.py:199
cloudra/cli.py:200      value = getattr(target, final_key)
cloudra/cli.py:201
cloudra/cli.py:202      # Mask keys
cloudra/cli.py:203      if "key" in key.lower() and isinstance(value, str) and value:
cloudra/cli.py:204          value = "***"
cloudra/cli.py:205
cloudra/cli.py:206      console.print(value)
cloudra/cli.py:207
```

```
cloudra/cli.py:208
cloudra/cli.py:209 @config_cmd.command(name="set")
cloudra/cli.py:210 @click.argument("key")
cloudra/cli.py:211 @click.argument("value")
cloudra/cli.py:212 def config_set(key, value):
cloudra/cli.py:213     """设置单个配置项。例如: cloudra config set coordinator.model gpt-4o"""
cloudra/cli.py:214     from cloudra.infra.config import set_config_value
cloudra/cli.py:215
cloudra/cli.py:216     try:
cloudra/cli.py:217         set_config_value(key, value)
cloudra/cli.py:218     except ValueError as exc:
cloudra/cli.py:219         console.print(f"[red]{exc}[/red]")
cloudra/cli.py:220         sys.exit(1)
cloudra/cli.py:221     console.print(f"[green]已设置 {key} = {value}[/green]")
cloudra/cli.py:222
cloudra/cli.py:223
cloudra/cli.py:224 @cli.command()
cloudra/cli.py:225 @click.argument("conv_id")
cloudra/cli.py:226 def logs(conv_id):
cloudra/cli.py:227     """查看某个对话的日志。"""
cloudra/cli.py:228     log_dir = paths.logs_dir(conv_id)
cloudra/cli.py:229
cloudra/cli.py:230     if not log_dir.exists():
cloudra/cli.py:231         console.print(f"[yellow]对话 {conv_id} 暂无日志。[/yellow]")
cloudra/cli.py:232         return
cloudra/cli.py:233
cloudra/cli.py:234     for log_file in sorted(log_dir.iterdir()):
cloudra/cli.py:235         if log_file.is_file():
cloudra/cli.py:236             console.print(Panel(f"[bold]{log_file.name}[/bold]", style="blue"))
cloudra/cli.py:237             try:
cloudra/cli.py:238                 with open(log_file, "r", encoding="utf-8") as f:
cloudra/cli.py:239                     content = f.read()
cloudra/cli.py:240                 console.print(content[:5000] if len(content) > 5000 else content)
cloudra/cli.py:241                 if len(content) > 5000:
cloudra/cli.py:242                     console.print("[dim]... (日志过长，已截断)[/dim]")
cloudra/cli.py:243             except Exception as e:
cloudra/cli.py:244                 console.print(f"[red]读取日志失败: {e}[/red]")
cloudra/cli.py:245             console.print()
cloudra/cli.py:246
cloudra/cli.py:247
cloudra/cli.py:248 @cli.command()
cloudra/cli.py:249 def skills():
cloudra/cli.py:250     """列出可用的 Skills。"""
cloudra/cli.py:251     from cloudra.base.skill_class import skill_loader
cloudra/cli.py:252
cloudra/cli.py:253     loader = skill_loader()
cloudra/cli.py:254     skills_data = loader.skill_datas_get()
cloudra/cli.py:255
cloudra/cli.py:256     if not skills_data:
cloudra/cli.py:257         console.print("[yellow]暂无可用 Skills。[/yellow]")
cloudra/cli.py:258         return
cloudra/cli.py:259
```

```
cloudra/cli.py:260      table = Table(title="可用 Skills", show_header=True, header_style="bold green")
cloudra/cli.py:261      table.add_column("名称", style="cyan")
cloudra/cli.py:262      table.add_column("描述")
cloudra/cli.py:263
cloudra/cli.py:264      for skill in skills_data:
cloudra/cli.py:265          table.add_row(
cloudra/cli.py:266              skill.get("name", "N/A"),
cloudra/cli.py:267              skill.get("description", "")[:80],
cloudra/cli.py:268          )
cloudra/cli.py:269
cloudra/cli.py:270      console.print(table)
cloudra/cli.py:271
cloudra/cli.py:272
cloudra/cli.py:273  @cli.group(name="backup")
cloudra/cli.py:274  def backup_cmd():
cloudra/cli.py:275      """创建、校验和恢复 Cloudra 数据备份。"""
cloudra/cli.py:276      pass
cloudra/cli.py:277
cloudra/cli.py:278
cloudra/cli.py:279  @backup_cmd.command(name="create")
cloudra/cli.py:280  @click.argument("destination", type=click.Path(path_type=Path))
cloudra/cli.py:281  def backup_create(destination: Path):
cloudra/cli.py:282      """在线快照 SQLite，并创建带 SHA-256 清单的备份。"""
cloudra/cli.py:283      from cloudra.infra.operations import BackupError, create_backup
cloudra/cli.py:284
cloudra/cli.py:285      try:
cloudra/cli.py:286          manifest = create_backup(Path(get_config().data_dir), destination)
cloudra/cli.py:287      except BackupError as exc:
cloudra/cli.py:288          raise click.ClickException(str(exc)) from exc
cloudra/cli.py:289      console.print(
cloudra/cli.py:290          f"[green]备份完成:[/green] {destination.resolve()} "
cloudra/cli.py:291          f"({len(manifest['files'])} files)"
cloudra/cli.py:292      )
cloudra/cli.py:293
cloudra/cli.py:294
cloudra/cli.py:295  @backup_cmd.command(name="verify")
cloudra/cli.py:296  @click.argument("archive", type=click.Path(exists=True, dir_okay=False, path_type=Path))
cloudra/cli.py:297  def backup_verify(archive: Path):
cloudra/cli.py:298      """校验备份清单、成员范围与全部 SHA-256。"""
cloudra/cli.py:299      from cloudra.infra.operations import BackupError, verify_backup
cloudra/cli.py:300
cloudra/cli.py:301      try:
cloudra/cli.py:302          manifest = verify_backup(archive)
cloudra/cli.py:303      except BackupError as exc:
cloudra/cli.py:304          raise click.ClickException(str(exc)) from exc
cloudra/cli.py:305      console.print(f"[green]备份有效[/green] ({len(manifest['files'])} files)")
cloudra/cli.py:306
cloudra/cli.py:307
cloudra/cli.py:308  @backup_cmd.command(name="restore")
cloudra/cli.py:309  @click.argument("archive", type=click.Path(exists=True, dir_okay=False, path_type=Path))
cloudra/cli.py:310  @click.argument("destination", type=click.Path(path_type=Path))
cloudra/cli.py:311  def backup_restore(archive: Path, destination: Path):
```

```
cloudra/cli.py:312      """恢复到不存在或空的目录；不会覆盖现有数据。"""
cloudra/cli.py:313      from cloudra.infra.operations import BackupError, restore_backup
cloudra/cli.py:314
cloudra/cli.py:315      try:
cloudra/cli.py:316          manifest = restore_backup(archive, destination)
cloudra/cli.py:317      except BackupError as exc:
cloudra/cli.py:318          raise click.ClickException(str(exc)) from exc
cloudra/cli.py:319      console.print(
cloudra/cli.py:320          f"[green]恢复完成:[/green] {destination.resolve()} "
cloudra/cli.py:321          f"({len(manifest['files'])} files)"
cloudra/cli.py:322      )
cloudra/cli.py:323
cloudra/cli.py:324
cloudra/cli.py:325  def main():
cloudra/cli.py:326      """CLI 入口函数。"""
cloudra/cli.py:327      cli()
cloudra/core/__init__.py:1  """cloudra.core 子包。"""
cloudra/core/agent.py:1 import asyncio
cloudra/core/agent.py:2 import hashlib
cloudra/core/agent.py:3 import inspect
cloudra/core/agent.py:4 import json
cloudra/core/agent.py:5 import os
cloudra/core/agent.py:6 from datetime import datetime
cloudra/core/agent.py:7 from typing import Any, Optional
cloudra/core/agent.py:8 from urllib.parse import urlsplit
cloudra/core/agent.py:9 from uuid import uuid4
cloudra/core/agent.py:10
cloudra/core/agent.py:11    from pydantic import BaseModel, Field, PrivateAttr, model_validator
cloudra/core/agent.py:12
cloudra/core/agent.py:13    from cloudra import __version__
cloudra/core/agent.py:14    from cloudra.base.tools_class import Tool
cloudra/core/agent.py:15    from cloudra.dag.models import OrchestrationMode, normalize_orchestration_mode
cloudra/core/agent.py:16    from cloudra.infra import events
cloudra/core/agent.py:17    from cloudra.infra.config import get_config
cloudra/core/agent.py:18    from cloudra.infra.llm_client import ChatResponse, LLMClient
cloudra/core/agent.py:19    from cloudra.infra.runtime_profile import filter_tools_for_runtime
cloudra/core/agent.py:20    from cloudra.infra.token_estimator import TokenEstimator
cloudra/core/agent.py:21    from cloudra.persistence.context_manager import SUPERVISOR_ADVICE_MARKER, ContextManager
cloudra/core/agent.py:22    from cloudra.persistence.database import ConversationDB
cloudra/core/agent.py:23    from cloudra.persistence.workspace import resolve_working_directory
cloudra/core/agent.py:24    from cloudra.runtime.prompt_assembler import (
cloudra/core/agent.py:25        PromptAssembler,
cloudra/core/agent.py:26        PromptAssembly,
cloudra/core/agent.py:27        load_role_prompt,
cloudra/core/agent.py:28    )
cloudra/core/agent.py:29    from cloudra.tools import ToolEntry, get_tools_lst
cloudra/core/agent.py:30    from cloudra.tools.core.permission_model import Permission, check_permission
cloudra/core/agent.py:31
cloudra/core/agent.py:32    # todo 提醒注入间隔（推理轮数）：每 N 轮提醒一次模型读取任务清单
cloudra/core/agent.py:33    _TODO_REMIND_EVERY = 10
cloudra/core/agent.py:34
cloudra/core/agent.py:35    # 模型给出结论但未调用 stop 时，注入一次目标确认提醒（一次性 force）
```

```
cloudra/core/agent.py:36    _GOAL_CONFIRM_REMINDER = (
cloudra/core/agent.py:37        "[System] You just stated a conclusion without calling the stop tool. "
cloudra/core/agent.py:38        "The task only ends after you call stop; prose alone leaves the loop spinning. "
cloudra/core/agent.py:39        "Use goal_set to reconfirm the objective and completion criteria, or todo_read to check the task list, "
cloudra/core/agent.py:40        "and decide whether the goal is actually met: if done, call stop with the final answer; "
cloudra/core/agent.py:41        "if not, keep going. If you already called stop and this round is only an event wakeup: "
cloudra/core/agent.py:42        "answer the user first if there is a user message; if there is no new work, call stop again to finish silently."
cloudra/core/agent.py:43    )
cloudra/core/agent.py:44
cloudra/core/agent.py:45    # 曾经 stop 过的实例被外部事件唤醒后，持续注入的复核轮提示（每轮都带，直到再次 stop）
cloudra/core/agent.py:46    _REVIVED_AFTER_STOP_NOTICE = (
cloudra/core/agent.py:47        "[System] This task already finished via the stop tool; this is a review round woken by an external event "
cloudra/core/agent.py:48        "(message/reminder). Answer the user first: if this round has a user question, instruction, report request, or stop request, "
cloudra/core/agent.py:49        "reply in prose immediately (user messages are not small talk; do not check compute, change reminders, or stop first). "
cloudra/core/agent.py:50        "A `role=user` `[Incoming call]` line also counts as a user ask: answer it before considering a silent stop. "
cloudra/core/agent.py:51        "After answering, if the user asked you to stop, finish as requested; if background work is still running and the user did not ask to stop, you may stop to wait. "
cloudra/core/agent.py:52        "Only when the wakeup is an expired reminder, a no-reply notice, or there is truly no user request should you call stop again "
cloudra/core/agent.py:53        "to finish silently (repeat stop is safe and recommended). Do not emit idle prose unrelated to the user's question."
cloudra/core/agent.py:54    )
cloudra/core/agent.py:55
cloudra/core/agent.py:56
cloudra/core/agent.py:57    def load_prompt(agent_name: str) -> str:
cloudra/core/agent.py:58        """兼容入口：读取角色 prompt（按 mtime 缓存，文件变更立即生效）。"""
cloudra/core/agent.py:59        return load_role_prompt(agent_name)
cloudra/core/agent.py:60
cloudra/core/agent.py:61
cloudra/core/agent.py:62    class Agent(BaseModel):
cloudra/core/agent.py:63        '''agent父类，用于定义各类agent属性：
cloudra/core/agent.py:64        1.唯一标识符（id）
cloudra/core/agent.py:65        2.agent名称(name)
cloudra/core/agent.py:66        3.agent类型(type)
cloudra/core/agent.py:67        4.agent使用模型（model）
cloudra/core/agent.py:68        5.agent使用模型url（llm_site）
cloudra/core/agent.py:69        6.agent使用模型key（_api_key）
cloudra/core/agent.py:70        7.agent使用模型温度（temperature）
cloudra/core/agent.py:71        8.agent使用模型最大token（max_tokens）
cloudra/core/agent.py:72        9.agent使用模型top_p（top_p）
cloudra/core/agent.py:73        10.agent使用模型频率惩罚（frequency_penalty）
cloudra/core/agent.py:74        11.agent使用模型存在惩罚（presence_penalty）
cloudra/core/agent.py:75        12.agent身份prompt（prompt）
cloudra/core/agent.py:76        13.agent上下文管理器（context_manager）
cloudra/core/agent.py:77        14.agent输出方式（is_stream）
cloudra/core/agent.py:78
cloudra/core/agent.py:79        该类的其他方法:
cloudra/core/agent.py:80        权限：has_write_permission，has_read_permission，script_run_permission，destructive_permission(均为布尔值输出)
cloudra/core/agent.py:81        llm_call方法：调用llm，输出ChatResponse或ChatStream
cloudra/core/agent.py:82        后续可能加入的方法：调用本地专用模型，不能通过openai接口，因此待定，可以上github上看看有没有开源的逆合成分析模型等
cloudra/core/agent.py:83        '''
cloudra/core/agent.py:84        model_config = {"arbitrary_types_allowed": True}
cloudra/core/agent.py:85
cloudra/core/agent.py:86        #唯一标识符
cloudra/core/agent.py:87        id: str
```

```
cloudra/core/agent.py:88
cloudra/core/agent.py:89        #agent名称
cloudra/core/agent.py:90        name: str
cloudra/core/agent.py:91
cloudra/core/agent.py:92        #agent类型：'coordinator' 或 'follower'
cloudra/core/agent.py:93        type: str
cloudra/core/agent.py:94
cloudra/core/agent.py:95        #agent使用模型
cloudra/core/agent.py:96        model: str
cloudra/core/agent.py:97
cloudra/core/agent.py:98        #agent使用模型url
cloudra/core/agent.py:99        llm_site: str
cloudra/core/agent.py:100
cloudra/core/agent.py:101       #agent使用模型key
cloudra/core/agent.py:102       _api_key: str
cloudra/core/agent.py:103
cloudra/core/agent.py:104       #agent使用模型温度
cloudra/core/agent.py:105       temperature: float = 0.7
cloudra/core/agent.py:106
cloudra/core/agent.py:107       #agent使用模型最大token（生成限制）
cloudra/core/agent.py:108       max_tokens: int
cloudra/core/agent.py:109
cloudra/core/agent.py:110       #agent使用模型上下文窗口（用于上下文压缩阈值计算）
cloudra/core/agent.py:111       context_window: int = 200000
cloudra/core/agent.py:112
cloudra/core/agent.py:113       #agent使用模型top_p
cloudra/core/agent.py:114       top_p: Optional[float] = None  # None = 不发送 top_p，用服务商默认值
cloudra/core/agent.py:115
cloudra/core/agent.py:116       #agent使用模型频率惩罚
cloudra/core/agent.py:117       frequency_penalty: float = 0.0
cloudra/core/agent.py:118
cloudra/core/agent.py:119       #agent使用模型存在惩罚
cloudra/core/agent.py:120       presence_penalty: float = 0.0
cloudra/core/agent.py:121
cloudra/core/agent.py:122       # 可选的调用方身份 prompt 覆盖；None 时由 prompt_name 在请求期加载。
cloudra/core/agent.py:123       prompt: Optional[str] = None
cloudra/core/agent.py:124
cloudra/core/agent.py:125       # prompts/{prompt_name}.md 的动态角色来源。
cloudra/core/agent.py:126       prompt_name: Optional[str] = None
cloudra/core/agent.py:127
cloudra/core/agent.py:128       #目标赋予prompt，如："你的任务是......"
cloudra/core/agent.py:129       query: str
cloudra/core/agent.py:130
cloudra/core/agent.py:131       is_stream: bool
cloudra/core/agent.py:132
cloudra/core/agent.py:133       # 当前 Agent 使用的模型是否支持视觉输入（多模态）
cloudra/core/agent.py:134       @property
cloudra/core/agent.py:135       def supports_vision(self) -> bool:
cloudra/core/agent.py:136           return False
cloudra/core/agent.py:137
cloudra/core/agent.py:138       # 可选的会话级唯一标识。子 Agent 被启动时会被赋予独立的 session_id，
cloudra/core/agent.py:139       # 用于在 ContextManager 中隔离同一 conversation 下的不同子任务上下文。
```

```
cloudra/core/agent.py:140       session_id: Optional[str] = None
cloudra/core/agent.py:141
cloudra/core/agent.py:142       # 上下文管理器：Agent 上下文的唯一真相来源
cloudra/core/agent.py:143       context_manager: Optional[ContextManager] = None
cloudra/core/agent.py:144
cloudra/core/agent.py:145       conversation_id: str
cloudra/core/agent.py:146
cloudra/core/agent.py:147       # API 会话可独立指定目录，避免修改进程级 cwd 导致并发任务互相影响。
cloudra/core/agent.py:148       # 构造后由 validator 收成绝对路径，不再保持 None。
cloudra/core/agent.py:149       working_directory: Optional[str] = None
cloudra/core/agent.py:150
cloudra/core/agent.py:151       orchestration_mode: OrchestrationMode = OrchestrationMode.STANDARD
cloudra/core/agent.py:152       orchestration_config: dict = Field(default_factory=dict)
cloudra/core/agent.py:153
cloudra/core/agent.py:154       # LLM 传输层客户端（实例级缓存，惰性构造）：
cloudra/core/agent.py:155       # 三级 fallback 的级别记忆（_curl_mode/_use_custom_headers）在其中维护
cloudra/core/agent.py:156       _llm_client: Optional[LLMClient] = None
cloudra/core/agent.py:157
cloudra/core/agent.py:158       # 工具 schema 列表缓存：工具集与本 Agent 权限在进程内是静态的，
cloudra/core/agent.py:159       # 无需每个推理步骤都重新实例化全部工具并重新生成 pydantic schema。
cloudra/core/agent.py:160       _tools_schema_cache: Optional[list] = None
cloudra/core/agent.py:161       _tools_schema_revision: Optional[tuple[str, ...]] = None
cloudra/core/agent.py:162       # 工具 schema 的 token 估算缓存：(revision, 名称列表) -> tokens，
cloudra/core/agent.py:163       # 随 schema 缓存一起失效（schema 不变则估算不变）。
cloudra/core/agent.py:164       _tools_schema_tokens_cache: Optional[tuple] = None
cloudra/core/agent.py:165
cloudra/core/agent.py:166       _last_prompt_assembly: Optional[PromptAssembly] = PrivateAttr(default=None)
cloudra/core/agent.py:167       _last_request_diagnostics: dict = PrivateAttr(default_factory=dict)
cloudra/core/agent.py:168       _dag_projection_provider: Any = PrivateAttr(default=None)
cloudra/core/agent.py:169
cloudra/core/agent.py:170       # todo 提醒节流：每 _TODO_REMIND_EVERY 轮注入一次；supervisor 反对时强制注入一次
cloudra/core/agent.py:171       _reasoning_round: int = 0
cloudra/core/agent.py:172       _todo_reminder_force: bool = False
cloudra/core/agent.py:173       # 模型给出结论但未调用 stop 时，一次性强制注入目标确认提醒
cloudra/core/agent.py:174       _goal_confirm_force: bool = False
cloudra/core/agent.py:175       # 本实例曾成功 stop、后被外部事件唤醒：每轮请求都带复核轮提示，引导模型再次 stop
cloudra/core/agent.py:176       _revived_after_stop: bool = False
cloudra/core/agent.py:177
cloudra/core/agent.py:178       @model_validator(mode="after")
cloudra/core/agent.py:179       def _resolve_working_directory(self) -> "Agent":
cloudra/core/agent.py:180           self.working_directory = resolve_working_directory(self.working_directory)
cloudra/core/agent.py:181           return self
cloudra/core/agent.py:182
cloudra/core/agent.py:183       def _init_context_manager(self) -> None:
cloudra/core/agent.py:184           '''初始化本 Agent 的 ContextManager。'''
cloudra/core/agent.py:185           self.context_manager = ContextManager(
cloudra/core/agent.py:186               agent_id=self.id,
cloudra/core/agent.py:187               conversation_id=self.conversation_id,
cloudra/core/agent.py:188               prompt=self.prompt,
cloudra/core/agent.py:189               query=self.query,
cloudra/core/agent.py:190               session_id=getattr(self, 'session_id', None),
cloudra/core/agent.py:191               on_compaction_progress=self._report_compaction_progress,
```

```
192 cloudra/core/agent.py:192           )
193 cloudra/core/agent.py:193
194 cloudra/core/agent.py:194       async def _report_compaction_progress(self, progress: dict) -> None:
195 cloudra/core/agent.py:195           """Broadcast non-durable compression progress without touching SQLite."""
196 cloudra/core/agent.py:196           await events.aemit(
197 cloudra/core/agent.py:197               self.conversation_id,
198 cloudra/core/agent.py:198               "context_compaction",
199 cloudra/core/agent.py:199               agent_id=self.id,
200 cloudra/core/agent.py:200               agent_name=self.name,
201 cloudra/core/agent.py:201               session_id=getattr(self, "session_id", None),
202 cloudra/core/agent.py:202               **progress,
203 cloudra/core/agent.py:203           )
204 cloudra/core/agent.py:204
205 cloudra/core/agent.py:205       def _ensure_context_manager(self) -> ContextManager:
206 cloudra/core/agent.py:206           '''确保上下文管理器已初始化；返回实例。'''
207 cloudra/core/agent.py:207           if self.context_manager is None:
208 cloudra/core/agent.py:208               self._init_context_manager()
209 cloudra/core/agent.py:209           return self.context_manager
210 cloudra/core/agent.py:210
211 cloudra/core/agent.py:211       async def get_context(self) -> list:
212 cloudra/core/agent.py:212           '''
213 cloudra/core/agent.py:213           加载上下文。ContextManager 是上下文的唯一真相来源。
214 cloudra/core/agent.py:214           '''
215 cloudra/core/agent.py:215           return await self._ensure_context_manager().load()
216 cloudra/core/agent.py:216
217 cloudra/core/agent.py:217       async def update_context(self, new_block: dict) -> None:
218 cloudra/core/agent.py:218           '''
219 cloudra/core/agent.py:219           更新上下文，新增一个块。
220 cloudra/core/agent.py:220           确保每条消息都有 role 字段。
221 cloudra/core/agent.py:221           文件路径根据 conversation_id 隔离。
222 cloudra/core/agent.py:222           '''
223 cloudra/core/agent.py:223           await self._ensure_context_manager().append(new_block)
224 cloudra/core/agent.py:224
225 cloudra/core/agent.py:225       async def clear_context(self) -> None:
226 cloudra/core/agent.py:226           '''清空上下文（通常用于任务结束后释放）。'''
227 cloudra/core/agent.py:227           await self._ensure_context_manager().clear()
228 cloudra/core/agent.py:228
229 cloudra/core/agent.py:229       def check_permission(self, tool: ToolEntry | Tool) -> bool:
230 cloudra/core/agent.py:230           if isinstance(tool, Tool):
231 cloudra/core/agent.py:231               tool_instance = tool
232 cloudra/core/agent.py:232           else:
233 cloudra/core/agent.py:233               tool_instance = tool.instantiate()
234 cloudra/core/agent.py:234           tool_perms = tool_instance.tool_permission_all()
235 cloudra/core/agent.py:235           agent_perms = self.permission_all()
236 cloudra/core/agent.py:236           return check_permission(agent_perms, tool_perms)
237 cloudra/core/agent.py:237
238 cloudra/core/agent.py:238
239 cloudra/core/agent.py:239       def get_available_tools(self, tools: Optional[list] = None) -> list:
240 cloudra/core/agent.py:240           """Return tools permitted for both this Agent and the runtime.
241 cloudra/core/agent.py:241
242 cloudra/core/agent.py:242           ``tools`` is injectable so the reAct dispatcher can apply the exact
243 cloudra/core/agent.py:243           same checks to its current registry snapshot.
```

```
cloudra/core/agent.py:244            """
cloudra/core/agent.py:245            tools = filter_tools_for_runtime(get_tools_lst() if tools is None else tools)
cloudra/core/agent.py:246            if normalize_orchestration_mode(self.orchestration_mode) is OrchestrationMode.STANDARD:
cloudra/core/agent.py:247                tools = [entry for entry in tools if not entry.tool_name.startswith("dag_")]
cloudra/core/agent.py:248            available_tools = []
cloudra/core/agent.py:249            for tool in tools:
cloudra/core/agent.py:250                if self.check_permission(tool):
cloudra/core/agent.py:251                    available_tools.append(tool)
cloudra/core/agent.py:252            return available_tools
cloudra/core/agent.py:253
cloudra/core/agent.py:254        def has_manage_permission(self) -> bool:
cloudra/core/agent.py:255            '''
cloudra/core/agent.py:256            启动子agent的权限。
cloudra/core/agent.py:257            '''
cloudra/core/agent.py:258            return False
cloudra/core/agent.py:259
cloudra/core/agent.py:260        def has_write_permission(self) -> bool:
cloudra/core/agent.py:261            '''
cloudra/core/agent.py:262            正常情况下，agent不允许写入文件，除非在子类中额外开放。
cloudra/core/agent.py:263            '''
cloudra/core/agent.py:264            return False
cloudra/core/agent.py:265
cloudra/core/agent.py:266        def has_read_permission(self) -> bool:
cloudra/core/agent.py:267            '''
cloudra/core/agent.py:268            检查是否有读取文件的权限。
cloudra/core/agent.py:269            '''
cloudra/core/agent.py:270            return True
cloudra/core/agent.py:271
cloudra/core/agent.py:272        def script_run_permission(self) -> bool:
cloudra/core/agent.py:273            '''
cloudra/core/agent.py:274            运行脚本的权限。
cloudra/core/agent.py:275            '''
cloudra/core/agent.py:276            return False
cloudra/core/agent.py:277
cloudra/core/agent.py:278        def destructive_permission(self) -> bool:
cloudra/core/agent.py:279            '''
cloudra/core/agent.py:280            如rm，mkdir等对目录结构进行变化的操作。
cloudra/core/agent.py:281            '''
cloudra/core/agent.py:282            return False
cloudra/core/agent.py:283
cloudra/core/agent.py:284        def permission_all(self) -> Permission:
cloudra/core/agent.py:285            perms = Permission.NONE
cloudra/core/agent.py:286            if self.has_read_permission():
cloudra/core/agent.py:287                perms |= Permission.READ
cloudra/core/agent.py:288            if self.has_write_permission():
cloudra/core/agent.py:289                perms |= Permission.WRITE
cloudra/core/agent.py:290            if self.has_manage_permission():
cloudra/core/agent.py:291                perms |= Permission.MANAGE
cloudra/core/agent.py:292            if self.script_run_permission():
cloudra/core/agent.py:293                perms |= Permission.SCRIPT
cloudra/core/agent.py:294            if self.destructive_permission():
cloudra/core/agent.py:295                perms |= Permission.DESTRUCTIVE
```

```
296 cloudra/core/agent.py:296           return perms
297 cloudra/core/agent.py:297
298 cloudra/core/agent.py:298       def _get_llm_client(self) -> LLMClient:
299 cloudra/core/agent.py:299           """返回实例级缓存的 LLM 传输层客户端（构造本身不做任何 I/O）。"""
300 cloudra/core/agent.py:300           if self._llm_client is None:
301 cloudra/core/agent.py:301               cfg = get_config()
302 cloudra/core/agent.py:302               role_cfg = cfg.coordinator if getattr(self, "type", "") == "coordinator" else cfg.
    follower
303 cloudra/core/agent.py:303               self._llm_client = LLMClient(
304 cloudra/core/agent.py:304                   api_key=self._api_key,
305 cloudra/core/agent.py:305                   base_url=self.llm_site,
306 cloudra/core/agent.py:306                   timeout=role_cfg.timeout,
307 cloudra/core/agent.py:307               )
308 cloudra/core/agent.py:308           return self._llm_client
309 cloudra/core/agent.py:309
310 cloudra/core/agent.py:310       def _role_llm_config(self):
311 cloudra/core/agent.py:311           """返回本 Agent 所属角色（coordinator/follower）的 LLM 配置。"""
312 cloudra/core/agent.py:312           cfg = get_config()
313 cloudra/core/agent.py:313           return cfg.coordinator if getattr(self, "type", "") == "coordinator" else cfg.follower
314 cloudra/core/agent.py:314
315 cloudra/core/agent.py:315       def _native_search_adapter(self):
316 cloudra/core/agent.py:316           """解析本 Agent 的模型原生网页搜索适配器（None = 关闭/不可用）。"""
317 cloudra/core/agent.py:317           from cloudra.infra.web_search_native import resolve_native_search
318 cloudra/core/agent.py:318
319 cloudra/core/agent.py:319           role_cfg = self._role_llm_config()
320 cloudra/core/agent.py:320           return resolve_native_search(
321 cloudra/core/agent.py:321               base_url=self.llm_site or "",
322 cloudra/core/agent.py:322               model=self.model or "",
323 cloudra/core/agent.py:323               mode=role_cfg.native_search,
324 cloudra/core/agent.py:324               params=role_cfg.native_search_params,
325 cloudra/core/agent.py:325           )
326 cloudra/core/agent.py:326
327 cloudra/core/agent.py:327       def _uses_responses_transport(self) -> bool:
328 cloudra/core/agent.py:328           """原生搜索需要走 Responses API（如 DeepSeek）时返回 True。"""
329 cloudra/core/agent.py:329           adapter = self._native_search_adapter()
330 cloudra/core/agent.py:330           return adapter is not None and adapter.transport == "responses"
331 cloudra/core/agent.py:331
332 cloudra/core/agent.py:332       def _todo_reminder(self) -> str:
333 cloudra/core/agent.py:333           """返回 todo 提醒文案；该 agent 未建过清单时为空串（纯查询，不做节流）。"""
334 cloudra/core/agent.py:334           try:
335 cloudra/core/agent.py:335               from cloudra.tools.Plantool._store import agent_store_path, load_json
336 cloudra/core/agent.py:336
337 cloudra/core/agent.py:337               if load_json(agent_store_path(self, "todos.json"), default=None):
338 cloudra/core/agent.py:338                   return (
339 cloudra/core/agent.py:339                       "[System] You are maintaining a todo list. Before this round, call "
340 cloudra/core/agent.py:340                       "todo_read to see the current list, confirm which item is in progress, then
     decide the next step; "
341 cloudra/core/agent.py:341                       "after finishing an item, immediately todo_write to update its status."
342 cloudra/core/agent.py:342                   )
343 cloudra/core/agent.py:343           except Exception:
344 cloudra/core/agent.py:344               pass
345 cloudra/core/agent.py:345           return ""
346 cloudra/core/agent.py:346
347 cloudra/core/agent.py:347       def _should_inject_todo_reminder(self) -> bool:
```

```
348 cloudra/core/agent.py:348           """todo 提醒注入节奏：每 _TODO_REMIND_EVERY 轮一次，或 supervisor
349 cloudra/core/agent.py:349           反对后强制注入一次（supervisor 反对是重要转向节点，模型需要重新
350 cloudra/core/agent.py:350           对照清单校正方向）。"""
351 cloudra/core/agent.py:351           self._reasoning_round += 1
352 cloudra/core/agent.py:352           force = self._todo_reminder_force
353 cloudra/core/agent.py:353           self._todo_reminder_force = False
354 cloudra/core/agent.py:354           return force or (self._reasoning_round % _TODO_REMIND_EVERY == 0)
355 cloudra/core/agent.py:355
356 cloudra/core/agent.py:356       def _has_goal_anchor(self) -> bool:
357 cloudra/core/agent.py:357           """该 agent 是否设过目标锚点（调用过 goal_set）。"""
358 cloudra/core/agent.py:358           try:
359 cloudra/core/agent.py:359               from cloudra.tools.Plantool._store import agent_store_path, load_json
360 cloudra/core/agent.py:360
361 cloudra/core/agent.py:361               return bool(load_json(agent_store_path(self, "goal.json"), default=None))
362 cloudra/core/agent.py:362           except Exception:
363 cloudra/core/agent.py:363               return False
364 cloudra/core/agent.py:364
365 cloudra/core/agent.py:365       def _goal_confirm_reminder(self) -> str:
366 cloudra/core/agent.py:366           """一次性目标确认提醒：仅当该 agent 设过目标锚点（goal_set）时有效。
367 cloudra/core/agent.py:367
368 cloudra/core/agent.py:368           reAct 在「无工具调用」轮置 force 后，下一轮请求注入——即首次出现
369 cloudra/core/agent.py:369           无工具轮时提醒模型确认 goal 是否完成、决定 stop 还是继续。
370 cloudra/core/agent.py:370           没设过 goal 时返回空串（无目标可确认，不注入）。
371 cloudra/core/agent.py:371           """
372 cloudra/core/agent.py:372           if not self._goal_confirm_force:
373 cloudra/core/agent.py:373               return ""
374 cloudra/core/agent.py:374           self._goal_confirm_force = False
375 cloudra/core/agent.py:375           return _GOAL_CONFIRM_REMINDER if self._has_goal_anchor() else ""
376 cloudra/core/agent.py:376
377 cloudra/core/agent.py:377       @staticmethod
378 cloudra/core/agent.py:378       def _tool_name_of(schema: dict) -> str:
379 cloudra/core/agent.py:379           function = schema.get("function", {}) if isinstance(schema, dict) else {}
380 cloudra/core/agent.py:380           return function.get("name") if isinstance(function, dict) else None
381 cloudra/core/agent.py:381
382 cloudra/core/agent.py:382       def _get_tools_schema(self) -> list[dict]:
383 cloudra/core/agent.py:383           """Cache schemas while invalidating when online device capabilities change."""
384 cloudra/core/agent.py:384           # Tests and embedders may explicitly inject a fixed schema list.
385 cloudra/core/agent.py:385           if (
386 cloudra/core/agent.py:386               self._tools_schema_cache is not None
387 cloudra/core/agent.py:387               and self._tools_schema_revision is None
388 cloudra/core/agent.py:388           ):
389 cloudra/core/agent.py:389               return self._tools_schema_cache
390 cloudra/core/agent.py:390           available_entries = self.get_available_tools()
391 cloudra/core/agent.py:391           revision = tuple(sorted(entry.tool_name for entry in available_entries))
392 cloudra/core/agent.py:392           if (
393 cloudra/core/agent.py:393               self._tools_schema_cache is None
394 cloudra/core/agent.py:394               or self._tools_schema_revision != revision
395 cloudra/core/agent.py:395           ):
396 cloudra/core/agent.py:396               available_tool_classes = []
397 cloudra/core/agent.py:397               for entry in available_entries:
398 cloudra/core/agent.py:398                   tool_instance = entry.instantiate(subagent=self)
399 cloudra/core/agent.py:399                   available_tool_classes.append(tool_instance.tool_schema_json())
```